



2021

서희정

PORTFOLIO

개발과 영상, 두 마리 토끼를 잡다.





서희정

SEO HEEJUNG



사진과 영상, 그리고 코딩을 좋아하는
돌아온 개발자입니다.

Github

<https://github.com/seoheejeong>

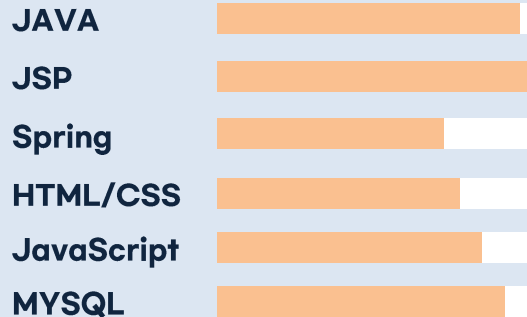
Instagram

https://www.instagram.com/seo_haligali

개인정보

생년월일 1994. 05. 20
전화번호 010-9937-4928
이메일 heejung0520@naver.com
학교/전공 장안대학교/멀티미디어콘텐츠과
희망 근무지 지역무관

Programing skill



주요 경력사항

2014 - 2017 장안대학교 멀티미디어콘텐츠과 졸업
2017 - 2019 두리웹 / 마케팅부서 재직
2019 - 2020 NHN / 클라우드 지원팀 재직
2020 - 2020 클래스101 / 101Production 재직

자격사항

2014 Office Excel® 2010
Office PowerPoint® 2010
2015 GTQ 일러스트 1급
2016 리눅스 마스터 2급
2019 자동차 운전면허 2종
ACA Photoshop cs6
ACA Premiere cs6
2021 COS PRO 2급 (Java)

CONTENTS

#01	JSP 졸업작품 -----	4p
#02	OOP CLASS DESIGN 프로젝트 -----	22p
#03	JAVA GUI 프로젝트 -----	30p
#04	JSP MVC2 구조 정리 -----	45p
#05	Spring Framework 프로젝트 -----	62p
#06	TOAST 사용자 가이드 -----	73p

* 항목을 클릭하면 해당 페이지로 이동할 수 있습니다.

#01

2016

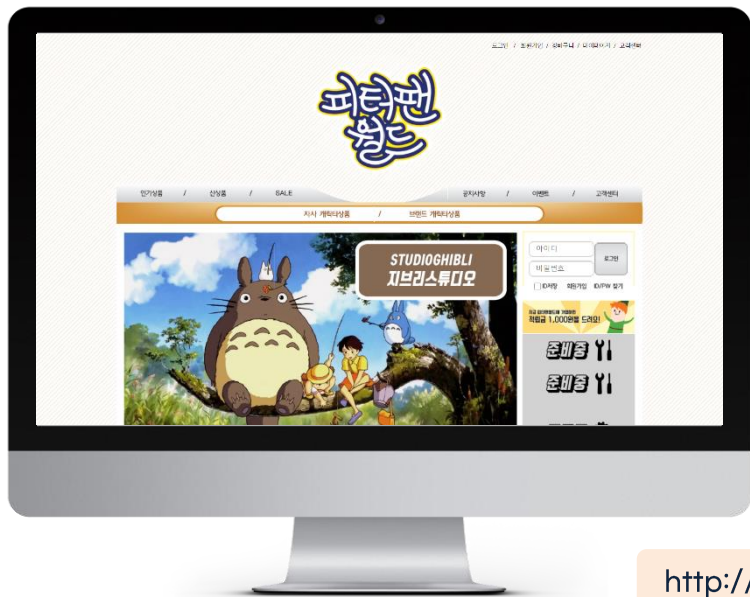
JSP 졸업작품

피터팬월드

피터팬 월드 (Peterpan world)

캐릭터 상품 판매 쇼핑몰

장난감을 좋아하는 어른들을 위한 키덜트샵을 주제로 작업한 웹 애플리케이션 모델링 방식 쇼핑몰 졸업작품입니다.



<http://peterpanworld.shop/>

개요

- 캐릭터 상품 판매를 위한 쇼핑몰 프로젝트
- 나를 위해 캐릭터 상품을 사는 어른들을 위한 서비스

개발 인원 및 기간

- 개발 기간 : 2016.08.10 ~ 2016.11.08
- 개발 인원 : FRONT-END 1명, BACK-END 1명

적용 기술

- JSP
- MYSQL
- HTML/CSS
- JavaScript
- JQuery

기여도

- 기획
- 개발

기여 부분

- 기획 및 데이터베이스 모델링
- JSP
- MYSQL
- JavaScript

핵심 기능 담당 역할

기획
데이터베이스 모델링
웹 퍼블리싱 (JavaScript, JQuery)
웹 서버 프로그래밍 (JSP, MYSQL)

1

JSP와 MYSQL을 사용하여 게시판 제작 (모델1 방식)

1. 상품 리스트
2. 장바구니
3. 공지사항
4. Q&A
5. 상품 카테고리
6. 회원정보

2

우편번호 찾기 서비스를 통해 주소 검색 시스템 구현

1. ZIPCODE 데이터베이스 사용
2. 도로명 주소의 데이터베이스를 찾지 못하여 지번 주소 데이터베이스를 사용하여 구현

핵심 기능 담당 역할

기획
데이터베이스 모델링
웹 퍼블리싱 (JavaScript, JQuery)
웹 서버 프로그래밍 (JSP, MYSQL)

3

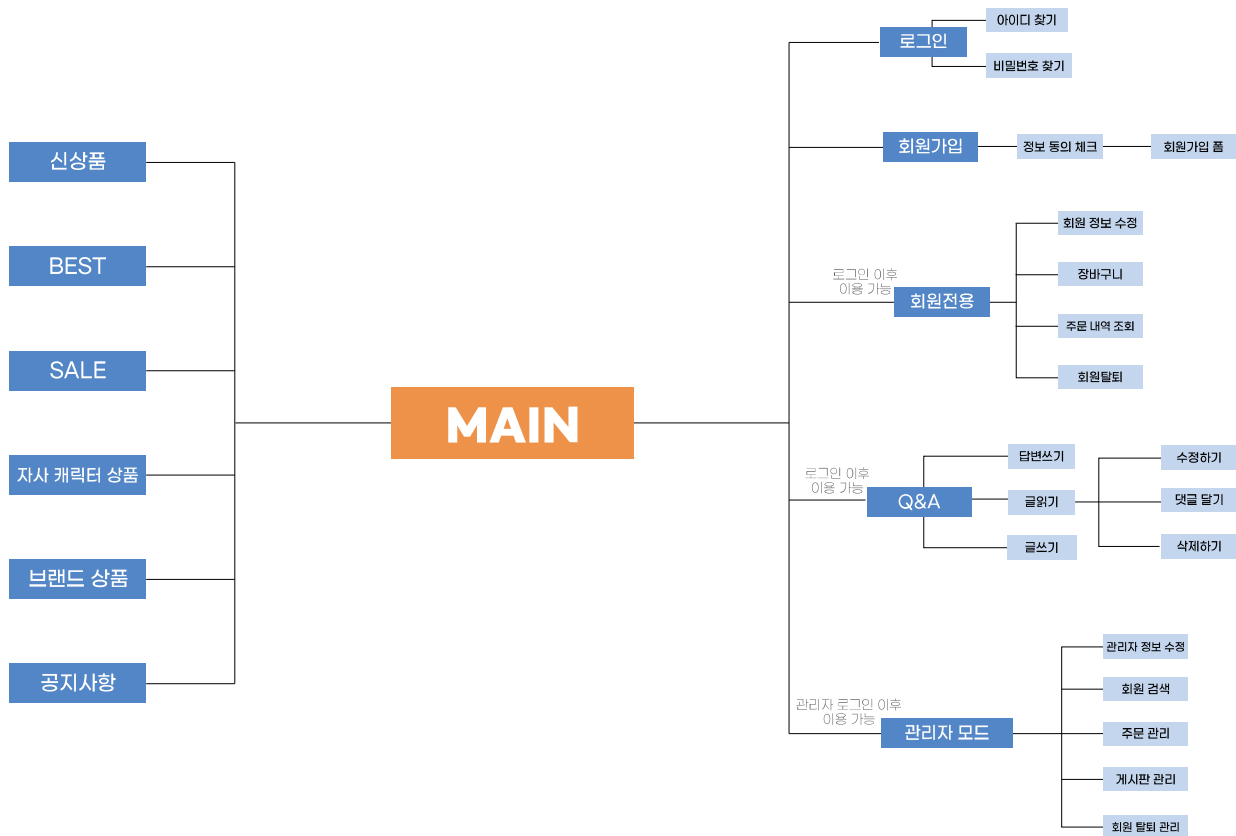
쇼핑몰에 필요한 데이터베이스 모델링

1. 회원정보 저장 테이블
2. 상품정보 저장 테이블
3. 장바구니 정보 저장 테이블
4. 주문정보 저장 테이블
5. 공지사항 테이블
6. Q&A 테이블

4

JQuery, JavaScript를 사용하여 웹 퍼블리싱

1. 메인 페이지 이미지 슬라이드 구현
2. 회원가입 유효성 검사 구현
3. 로그인 시 아이디 저장 구현



테이블명	member		
NO	Field	Type	Length
1	mem_num	INT	
2	userid *	VARCHAR	15
3	passwd	VARCHAR	15
4	usernم	VARCHAR	15
5	birthyear	VARCHAR	5
6	birthmonth	VARCHAR	5
7	birthdate	VARCHAR	5
8	gender	VARCHAR	7
9	zipcode	VARCHAR	8
10	addr1	VARCHAR	300
11	addr2	VARCHAR	300
12	telno1	VARCHAR	4

테이블명	member		
NO	Field	Type	Length
13	telno2	VARCHAR	5
14	telno3	VARCHAR	5
15	mail	VARCHAR	40
16	mailrcv	VARCHAR	2
17	milleage	DECIMAL	(7,0)
18	grade	VARCHAR	15

테이블명	admin		
NO	Field	Type	Length
1	admin_id	VARCHAR	15
2	admin_pw	VARCHAR	20
3	admin_name	VARCHAR	15

테이블명	Category1 (대분류)		
NO	Field	Type	Length
1	cat1cd *	VARCHAR	30
2	cat1nm	VARCHAR	30

테이블명	Category2 (중분류)		
NO	Field	Type	Length
1	cat2num *	INT	
2	cat1cd	VARCHAR	30
3	cat2cd	VARCHAR	30
4	cat2nm	VARCHAR	30

테이블명	goodsinfo		
NO	Field	Type	Length
1	cat1cd	VARCHAR	30
2	cat2cd	VARCHAR	30
3	goodsacd *	VARCHAR	20
4	goodsnm	VARCHAR	100
5	unitprice	DECIMAL	(7,0)
6	best_yn	CHAR	1
7	New_yn	CHAR	1
8	Sale_yn	CHAR	1
9	goodsimg1	VARCHAR	100
10	useflag	CHAR	1
11	regdtm	DATETIME	

테이블명	cart		
NO	Field	Type	Length
1	idx *	INT	
2	userid	VARCHAR	15
3	goodscd	VARCHAR	20
4	goodsnm	VARCHAR	100
5	unitprice	DECIMAL	(7,0)
6	qty	TINYINT	1
7	chkYN	CHAR	1

테이블명	order_D		
NO	Field	Type	Length
1	ordNo *	CHAR	12
2	ordSeq	TINYINT	1
3	goodsCd	CHAR	8
4	userid	VARCHAR	15
5	unitprice	DECIMAL	(7,0)
6	ordQty	TINYINT	1
7	deliUserName	VARCHAR	20
8	delizipcode	VARCHAR	8
9	deliaddr1	VARCHAR	300
10	deliaddr2	VARCHAR	300
11	delitelno1	VARCHAR	4

테이블명	order_D		
NO	Field	Type	Length
12	delitelno2	VARCHAR	5
13	delitelno3	VARCHAR	5
14	cancelYN	CHAR	1
15	ordAmt	DECIMAL	(7,0)

테이블명	order_H		
NO	Field	Type	Length
1	ordNo *	CHAR	10
2	ordDtm	DATETIME	
3	userid	VARCHAR	15
4	ordTamt	DECIMAL	(8,0)
5	zipcode	VARCHAR	8
6	addr1	VARCHAR	300
7	addr2	VARCHAR	300
8	telNo1	VARCHAR	4
9	telNo2	VARCHAR	5
10	telNo3	VARCHAR	5
11	deliUserName	VARCHAR	20
12	delizipcode	VARCHAR	8

테이블명	order_H		
NO	Field	Type	Length
13	deliaddr1	VARCHAR	300
14	deliaddr2	VARCHAR	300
15	delitelNo1	VARCHAR	4
16	delitelNo2	VARCHAR	5
17	delitelNo3	VARCHAR	5
18	cancelYN	CHAR	1
19	bankinfo	VARCHAR	100
20	canceldtm	DATETIME	
21	depositOK	VARCHAR	30
22	ordState	VARCHAR	30
23	deliver	VARCHAR	30
24	sendday	VARCHAR	30
25	recvday	VARCHAR	30

테이블명	notice		
NO	Field	Type	Length
1	num *	INT	
2	title	VARCHAR	50
3	contents	TEXT	
4	writer	VARCHAR	50
5	writedtm	DATE	
6	updatedtm	DATE	
7	readcnt	DECIMAL	(8,0)

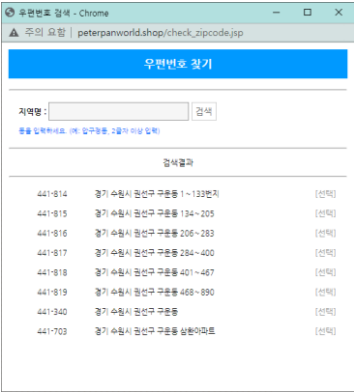
테이블명	QnA		
NO	Field	Type	Length
1	num *	INT	
2	mgrp	INT	
3	mseq	INT	
4	mlvl	INT	
5	userid	VARCHAR	15
6	title	VARCHAR	100
7	contents	TEXT	
8	lock_yn	CHAR	1
9	pwd	VARCHAR	15
10	writer	VARCHAR	50
11	writedtm	DATE	
12	updatedtm	DATE	
13	readcnt	DECIMAL	(8,0)



메인화면



회원가입



우편번호 검색

로그인 / 회원가입 / 장바구니 / 마이페이지 / 고객센터

피터팬 월드

인기상품 / 신상품 / SALE 공지사항 / 이벤트 / 고객센터

자녀 캐릭터상품 / 생일도 캐릭터상품

아이디
비밀번호

☐ 비밀번호 숨기기

로그인하기

*아이디 비밀번호를 잊으셨나요?
초등학생부터 초등학생이 아닌 모든 어린이가 신상정보를 입력하지 않으면서도, 부모님들이 신청하는 인증 전화번호를 입력해 주실 수 있습니다.

회원가입

*아이디/비밀번호가 기억나지 않으셨나요?
아이디 찾기 **비밀번호 찾기**

CALL CENTER
010-9997-4928
평일 10:00~18:00
주말, 공휴일 휴무

BANK INFO
계좌주: 피터팬월드
계좌번호: 123456-12-123456
계좌번호: 123456-12-1234-56
계좌번호: 123-123456-12-123

ABOUT
회사소개
이벤트안내
개인정보취급방침
이용약관

상호명: 피터팬 월드 / 대표: 김철수, 대표이사: 김철수 / 사업자등록번호: 123-12-12345 / 통신판매업 신고: 2018-경기분당-1234호 / 개인정보취급방침: 제1회
전화: 010-9997-4928 / 팩스: 010-9997-4929 / 이메일: you@peterpan.com / 주소: 경기도 성남시 분당구 분당로 112-1 (분당동) 4층 404호
Copyright © 피터팬월드 All rights reserved.

로그인

아이디 / 비밀번호 찾기

아이디 찾기 비밀번호 찾기

• 등록 정보로 찾기
회원 가입시 등록한 정보로 찾을 수 있습니다.

☒ 이메일 ☐ 휴대폰

이름:

아이디:

확인

☐ 이메일 ☒ 휴대폰

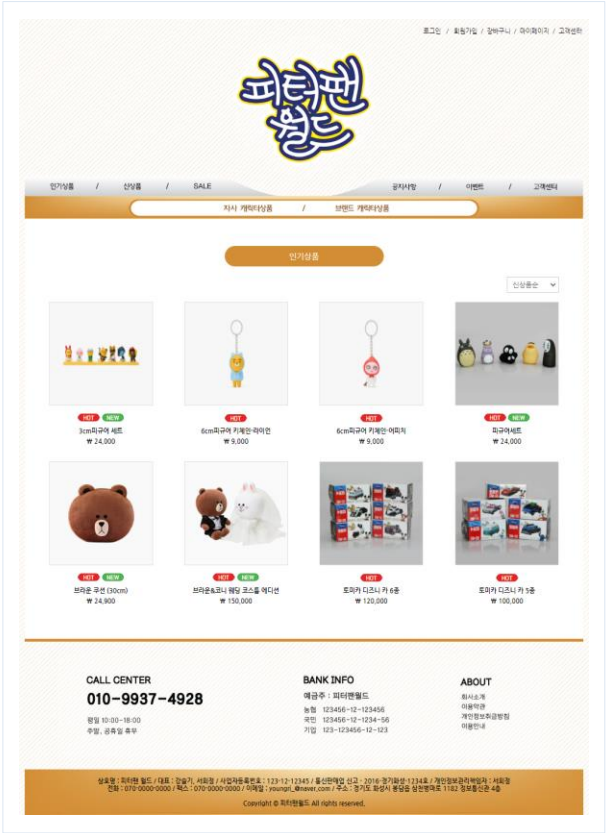
이름:

아이디:

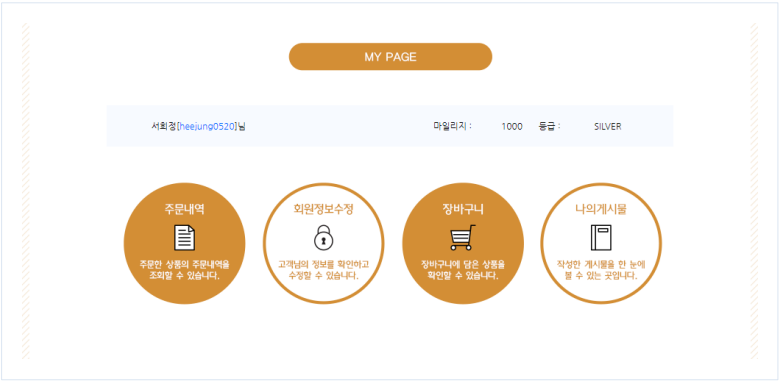
휴대폰: 010 - - -

확인

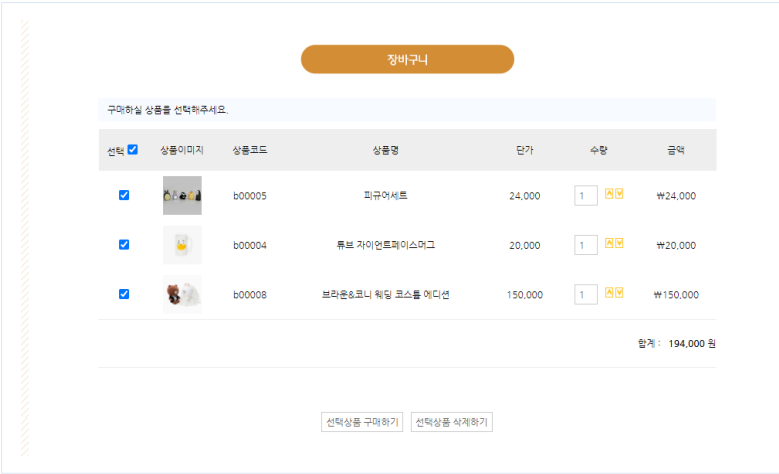
아이디/비밀번호 찾기



상품화면



마이페이지



장바구니

주문하기

주문 상품

구입 상품명	단가	수량	금액
 브라콘소니 해당 코스를 아디션	190,000	1	190,000
 흥보 자이언트라이스프로그	20,000	1	20,000
 피규어세트	24,000	1	24,000

합계 | 194,000 원

주문자 정보

성명

서희정

주민번호

441-919-무관번호 숨김

입주주소

경기 수원시 동탄구 우문동

상사주소

888번지

연락처

010-9937-4928

배송자 정보

성명

주민번호

무관번호 숨김

입주주소

상사주소

연락처

010- - -

주문메시지

(*)상품수가 기준 80개, 한글 기준 30개까지 입력 가능합니다.

결제 정보

결제방법

☒ 무통장 입금
 ☐ 신용카드

결제할 카드

신용카드

주문하기

재구매하기

주문화면

로그아웃 / 관리자 / 관리자 / 고객센터

피터팬 월드

공지사항

SALE

이벤트

고객센터

자사 캐릭터상품

선반인 캐릭터상품

고객센터

자주 묻는 질문

입금문의

배송문의

교환 / 반품문의

배송진행경 / 취소문의

입금확인여부 안내(2020.12.28)

입금확인여부 문의시 주문이완료 후 확인합니다. 기고된시점
 1. 주문하신 후 10일 이내 입금확인이 불가할경우 상품주문취소처리가됩니다.
 2. 주문하신 상품과 동일하지않은 상품이 배송될 경우 상품주문취소처리가됩니다.
 3. 주문하신 상품과 동일하지않은 상품이 배송될 경우 상품주문취소처리가됩니다.
 4. 주문하신 상품과 동일하지않은 상품이 배송될 경우 상품주문취소처리가됩니다.
 5. 입금확인이 불가할경우 상품주문취소처리가됩니다.

1:1 문의사항

자주 묻는 질문을 확인한 후에도 아직 궁금한 점이 남아있다면
 1:1 문의를 이용해 좀 더 자세한 질문을 해주세요.

1:1 문의하기

NO

SUBJECT

WRITE

DATE

1

입금확인 부탁드립니다.

서희정

2020-12-28

[1]

제목

답변

CALL CENTER

010-9937-4928

BANK INFO

예금주 : 피터팬월드

농협 123456-12-123456

국민 123456-12-123456

기타 123456-12-123

ABOUT

회사소개

이용약관

개인정보처리방침

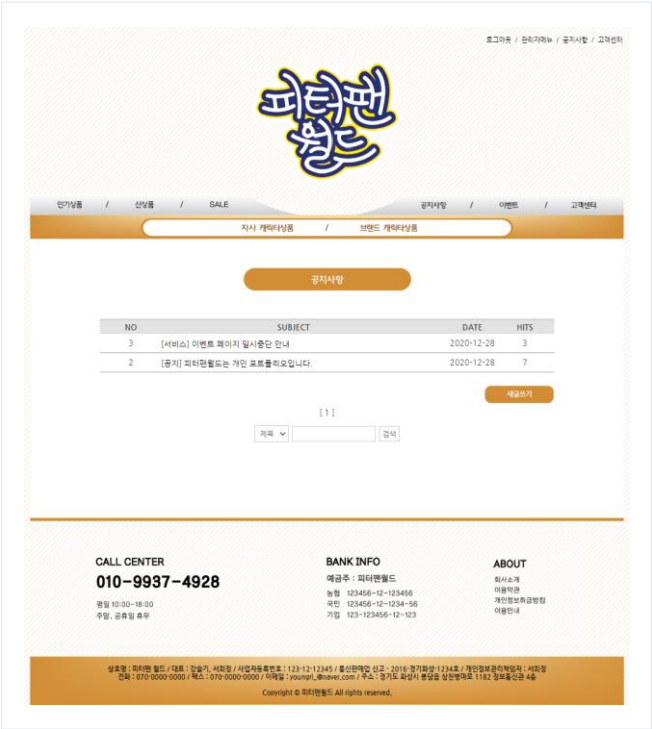
이메일주소

상호명 : 피터팬 월드 / 대표 : 김철수 / 사업자등록번호 : 12345678901234 / 통신판매업 신고 : 2019-경기분당-1234호 / 개인정보처리방침 : 해당없음 / 연락처 : 010-9937-4928 / 팩스 : 010-9937-4929 / 이메일 : peter@peter.com / 주소 : 경기도 수원시 동탄구 우문동 111-1, 112, 113호 1층 111호

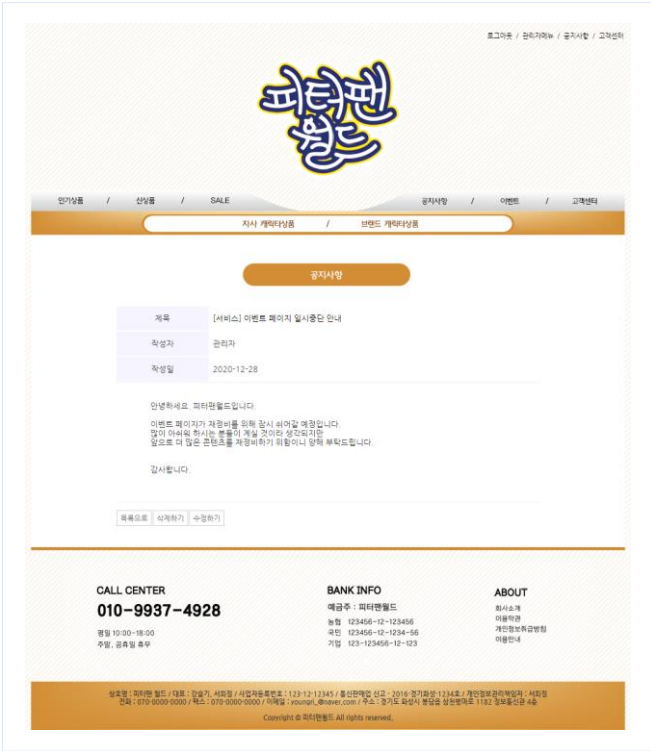
Copyright © 피터팬월드 All rights reserved.

고객센터

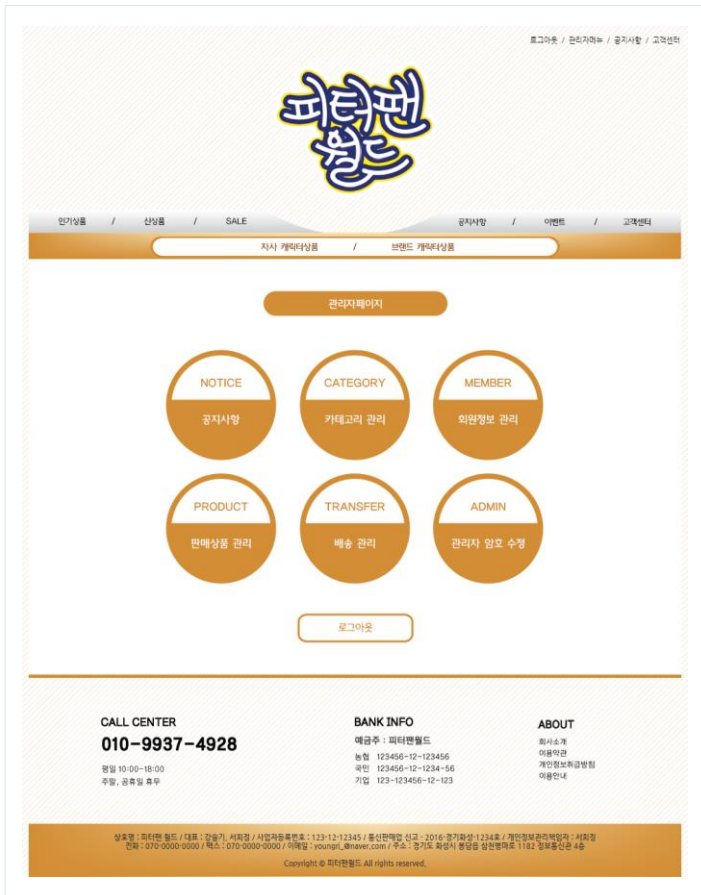
18



게시판



공지사항



관리자
페이지



카테고리 관리



배송관리

주요화면

#1 피터팬 월드

판매상품관리

대분류	중분류	상품코드	상품명	단가	인가	신상	세일	등록일	삭제
brand	토미카	b00011	토미카 디즈니 카 5종	90,000	N	N	Y	2020-12-21	삭제
brand	토미카	b00010	토미카 디즈니 카 5종	100,000	Y	N	N	2020-12-21	삭제
brand	토미카	b00009	토미카 디즈니 카 6종	120,000	Y	N	N	2020-12-21	삭제
brand	카카오포렌즈	b00001	3cm 피규어 세트	24,000	Y	Y	Y	2020-12-21	삭제
brand	지브리	b00005	피규어 세트	24,000	Y	Y	Y	2020-12-21	삭제
brand	라인프렌즈	b00008	브라운&로니 워딩 코스튬 에디션	150,000	Y	Y	N	2020-12-21	삭제
brand	라인프렌즈	b00007	브라운 후선 (30cm)	24,900	Y	Y	N	2020-12-21	삭제
brand	라인프렌즈	b00006	라인프렌즈 SD 미니 피규어 세트	54,000	N	N	N	2020-12-21	삭제
brand	카카오포렌즈	b00004	특별 지어언트페이스마스크	20,000	N	Y	N	2020-12-21	삭제
brand	카카오포렌즈	b00003	6cm 피규어 키체인-여피지	9,000	Y	N	N	2020-12-21	삭제
brand	카카오포렌즈	b00002	6cm 피규어 키체인-라이언	9,000	Y	N	N	2020-12-21	삭제

신규추가

대분류 검색

판매상품관리

판매상품관리

대분류

보행도 캐릭터상품

중분류

토미카

상품명

토미카 디즈니 카

판매단가

60000

인가상품여부

☐

신상상품여부

☐

이미지

파일 선택

toml_01.png

등록

취소

판매상품등록

판매상품관리

대분류

보행도 캐릭터상품

중분류

토미카

상품코드

b00012

상품명

토미카 디즈니 카

판매단가

60000

인가상품여부

☒ N ☐

신상상품여부

☐ N ☒

세일상품여부

☒ N ☐

변경전 이미지

변경후 이미지

파일 선택

선택된 파일 없음

수정

취소

판매상품수정

#02

2017

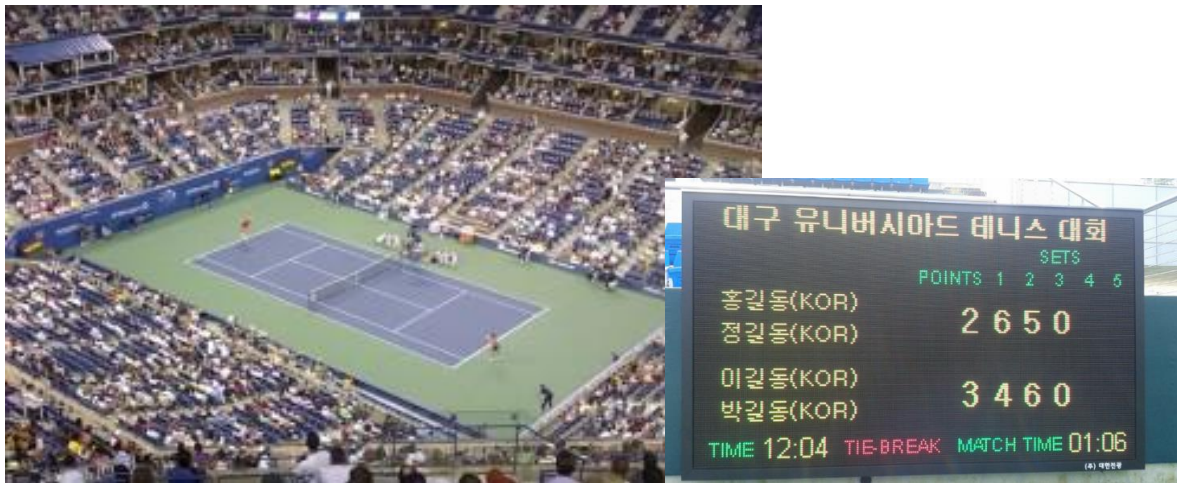
OOP CLASS DESIGN 프로젝트

테니스 스코어보드

테니스 스코어보드

콘솔 응용 프로그램으로 제작한 테니스 스코어보드

객체지향 프로그래밍의 특징을 활용하여 작업한 프로젝트입니다.



<https://github.com/seoheejung/TennisScoreBoard>

개요

- 테니스 경기의 특징을 계수하는 스코어보드 제작 프로젝트

개발 인원 및 기간

- 개발 기간 : 2017.04.14 ~ 2017.04.16
- 개발 인원 : 1명

적용 기술

- JAVA

기여도

- 기획
- 개발

OOP

- 추상화
- 상속
- 캡슐화

1

필수 메서드

1. pointWinner(Player[] players, int p)
 - p의 값이 1인가 2인가에 따라 해당 선수의 포인트를 증가시킨다.
 - 포인트 득점에 따른 게임 스코어 및 세트 스코어 계산은 테니스 규칙으로 정한다.
2. dispScoreBoard()
 - 현재의 스코어보드를 화면에 출력한다.

2

경기규칙

1. 두 선수 중 포인트를 득점한 선수의 게임 포인트를 증가시킨다.
득점할 때마다 득점 포인트는 0→15→30→40 순으로 표현한다.
2. 게임 점수가 40인 선수가 포인트를 획득하면 그 게임을 승리한다.
단, 두 선수가 모두 40점인 경우는 듀스가 된다.
듀스가 된 후에는 상대방보다 두 포인트를 더 획득한 선수가 그 게임을 승리한다.
3. 상대방보다 2게임 이상 차이를 두고 6게임 이상을 먼저 획득한 선수가 그 세트를 승리한다.
4. 경기 세트 수의 반 이상을 먼저 승리한 선수가 최종 승자이다.

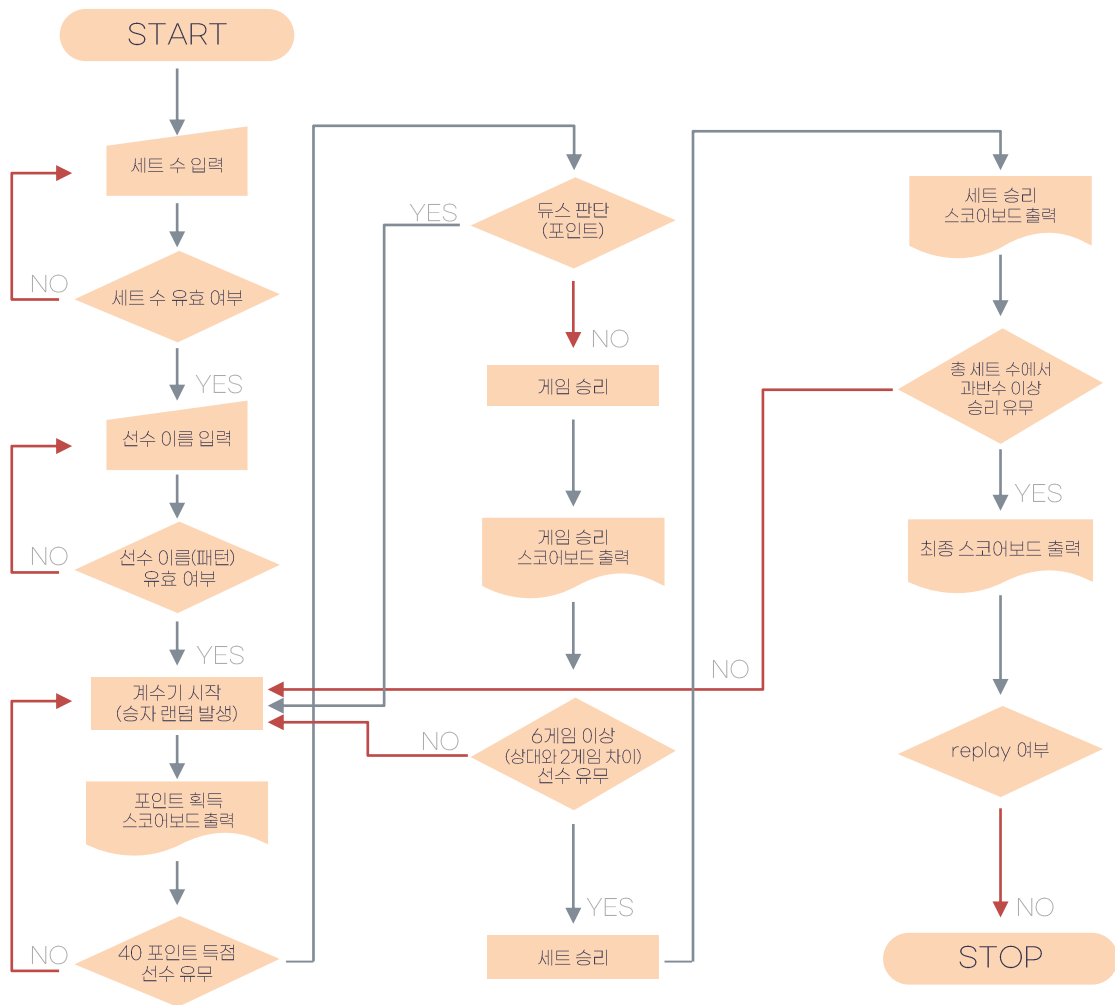
3

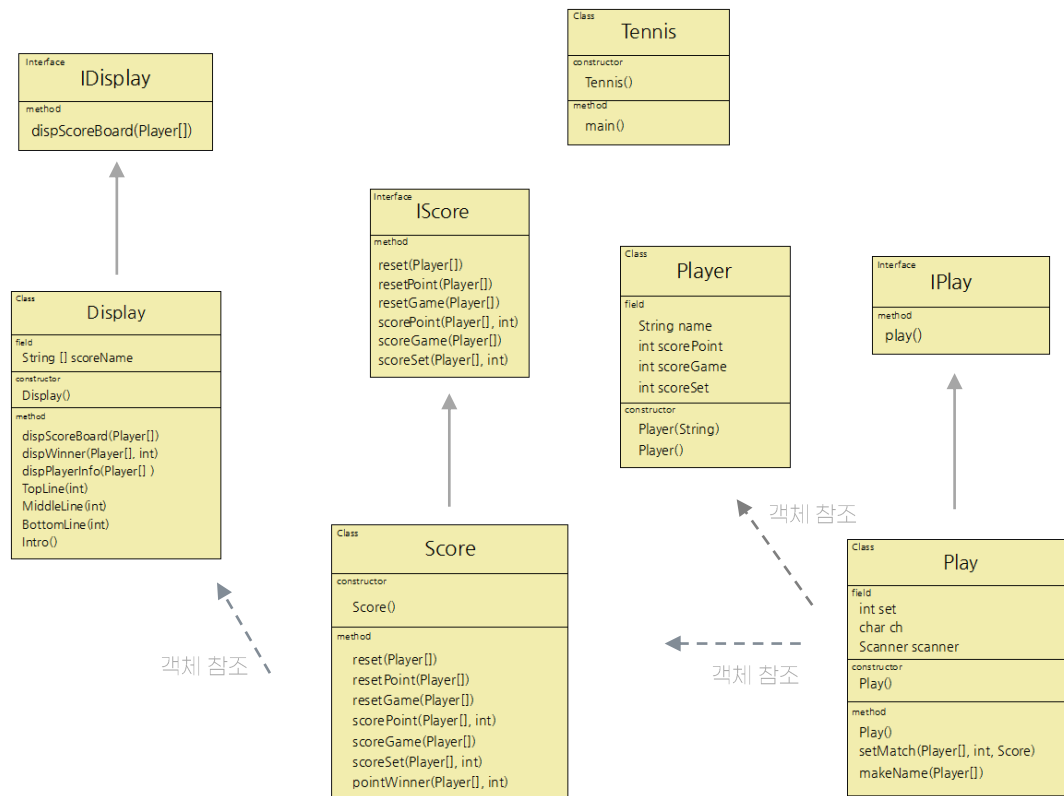
타이브레이크 적용 X

4

프로그램 동작 형식

1. 콘솔에서 경기의 세트 수, 두 선수의 이름을 입력한다.
2. 포인트를 획득한 선수를 입력받은 후, 스코어보드를 계산하여 출력하는 과정을
경기가 끝나 최종 승리자가 결정될 때까지 반복한다.
3. 상대방보다 2게임 이상 차이를 두고 6게임 이상을 먼저 획득한 선수가 그 세트를 승리한다.
4. 승자가 결정되면 최종 스코어보드와 함께 승자의 이름을 출력한다.





Class	Tennis
constructor	Tennis()
method	main()

메인 함수를 통해 게임을 실행하기 위한 클래스

Class	Player
field	String name int scorePoint int scoreGame int scoreSet
constructor	Player(String) Player()

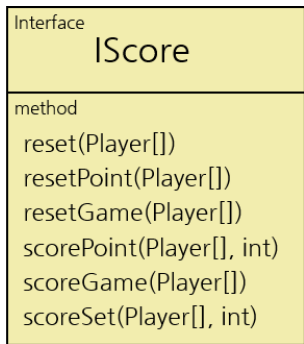
선수의 정보를 저장하는 클래스
(이름, 포인트 점수, 게임 점수, 세트 점수)

Interface	IPlay
method	play()

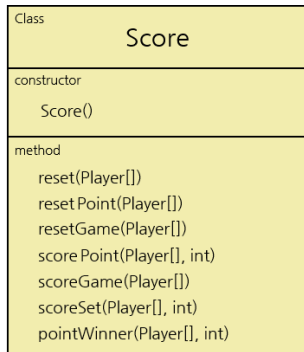
실행 인터페이스

Class	Play
field	int set char ch Scanner scanner
constructor	Play()
method	Play() setMatch(Player[], int, Score) makeName(Player[])

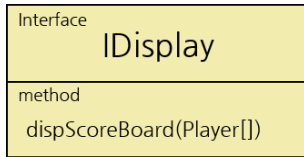
실행 인터페이스를 구현한 클래스
Player, Score 객체 참조
(실행, 세트매치, 선수 이름 선언 메소드)



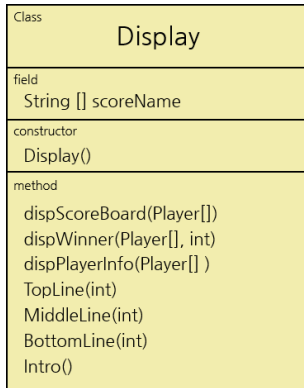
점수 계산 인터페이스



점수 계산 인터페이스를 구현한 클래스
Display 객체 참조
(리셋, 점수 계산 메소드)



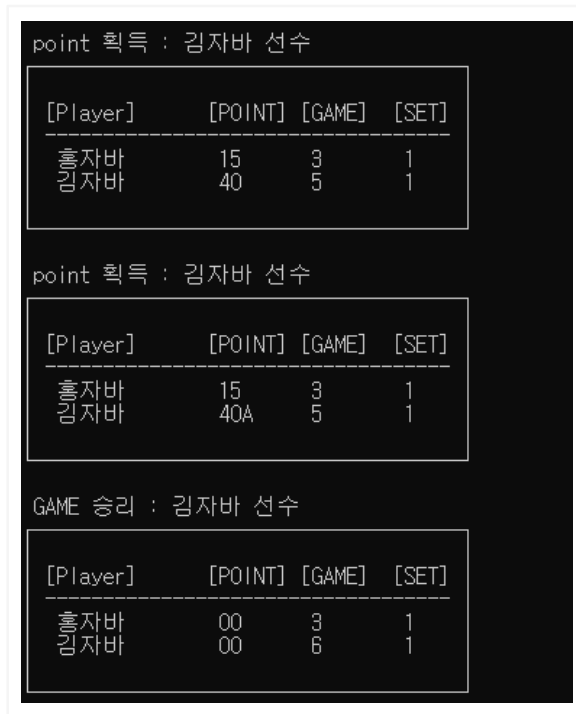
스코어 보드 출력 인터페이스



스코어 보드 출력 인터페이스를 구현한 클래스
(현재 스코어보드, 최종 스코어보드,
선수 정보 출력, 라인 그리기 메소드 등)



시합 시작 & Point 획득



Point 획득 & Game 승리



Set 승리 & 시합 종료

#03

2020

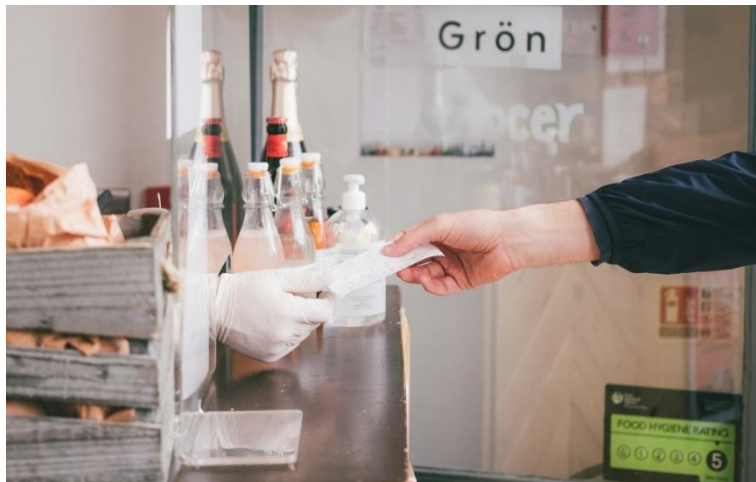
JAVA GUI 프로젝트

스터디카페 키오스크

스터디카페 키오스크

JAVA GUI를 활용한 키오스크 프로그램

키오스크의 일반화와 무인 계산 도입 확대, 스터디카페 증가 추세로 선택한 프로젝트입니다.



https://github.com/seoheejung/Swimpyo_Kiosk

<https://youtu.be/mjwVCuhgY5Y>

개요

- 스터디카페에서 사용되는 키오스크 제작 프로젝트
- 스터디카페를 창업하는 사장님을 타겟으로 한 서비스

개발 인원 및 기간

- 개발 기간 : 2020.12.3 ~ 2020.12.14
- 개발 인원 : 1명

적용 기술

- JAVA GUI

기여도

- 기획
- 개발
- 디자인



1. 로그인

회원만 스터디카페 이용 가능

2. 회원가입

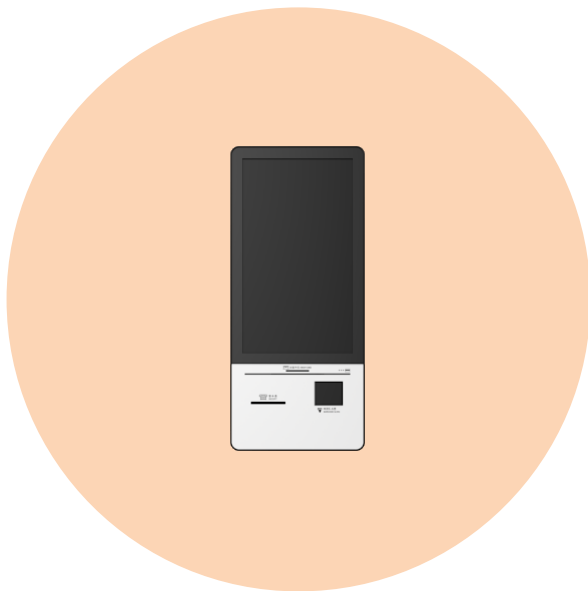
스터디카페 이용을 위한 가입 절차

3. 좌석 선택

이용 가능한 좌석만 선택 가능

4. 정보 수정

회원의 비밀번호 수정 가능



5. 기간 선택

시간권, 정액권 선택 가능

6. 결제

가상 현금결제 상황 구현

7. 퇴실 기능

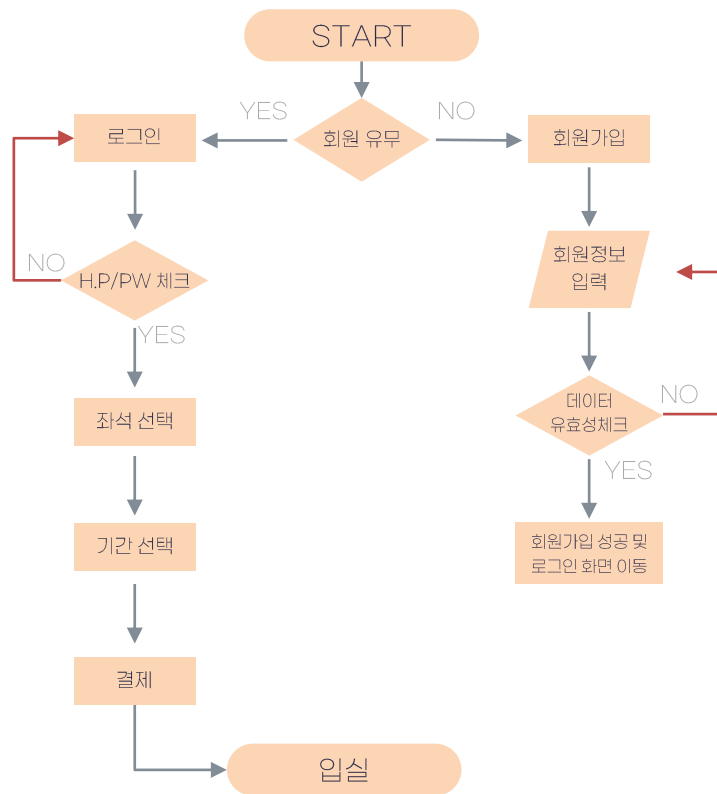
자리를 이용하고 있을 때만 퇴실 가능

8. 관리자 모드

사용자 정보, 좌석 이용 현황, 강제 퇴실

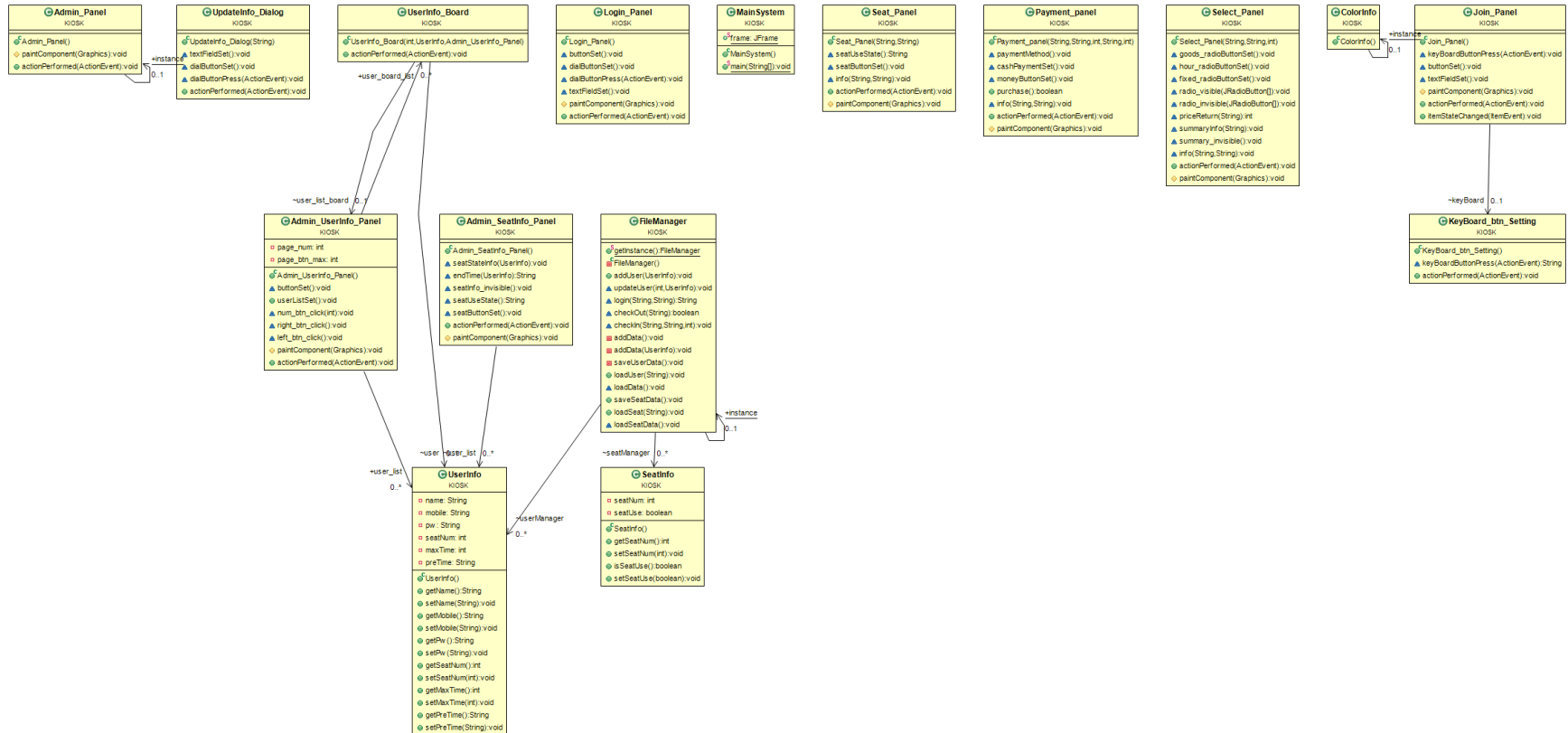
· 메인 시나리오 선정

- 1) 키오스크 앞에 선다
- 2) 로그인 or 회원가입
- 3) 빈 좌석 선택
- 4) 기간 선택
- 5) 결제 방법 선택
- 6) 결제 후 입실



UML 다이어그램

#3 스터디카페 키오스크



로그인

회원가입

좌석선택

기간선택

결제

정보수정



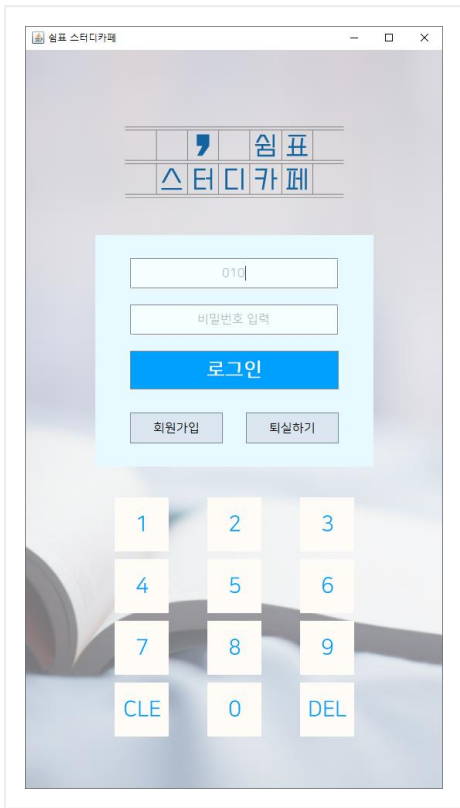
관리자모드



사용자정보



좌석이용현황



```
@Override
public void actionPerformed(ActionEvent e) {
    dialButtonPress(e);

    // 로그인 버튼을 눌렀을 때
    if (e.getSource() == login_btn) {
        if (mobile_tf.getText().equals("01000000000") && pw_tf.getText().equals("0000")) {
            MainSystem.frame.setContentPane(new Admin_Panel()); // 관리자모드 이동
            MainSystem.frame.revalidate();
        } else {
            String log = FileManager.instance.login(mobile_tf.getText(), pw_tf.getText());
            if (log.equals("")) {
                JOptionPane.showMessageDialog(null, "존재하지 않는 번호입니다.", "Message", JOptionPane.WARNING_MESSAGE);
            } else {
                MainSystem.frame.setContentPane(new Seat_Panel(log, mobile_tf.getText()));
                MainSystem.frame.revalidate();
            }
        }
    }

    // 회원가입 버튼을 눌렀을 때
    } else if (e.getSource() == join_btn) {
        MainSystem.frame.setContentPane(new Join_Panel());
        MainSystem.frame.revalidate();
    }

    // 퇴실하기 버튼을 눌렀을 때
    } else if (e.getSource() == checkOut_btn) {
        String log = FileManager.instance.login(mobile_tf.getText(), pw_tf.getText());
        if (log.equals("")) {
            JOptionPane.showMessageDialog(null, "존재하지 않는 번호입니다.", "Message", JOptionPane.WARNING_MESSAGE);
        } else {
            if (FileManager.instance.checkOut(mobile_tf.getText()) {
                JOptionPane.showMessageDialog(null, "퇴실이 완료되었습니다.", "Message", JOptionPane.INFORMATION_MESSAGE);
                MainSystem.frame.setContentPane(new Login_Panel());
                MainSystem.frame.revalidate();
            }
        }
    }
}

// 로그인
String login(String mobile, String pw) {
    String check_id = "";
    for (int i = 0; i < userManager.size(); i++) {
        if (mobile.equals(userManager.get(i).getMobile()) && pw.equals(userManager.get(i).getPw())) {
            check_id = userManager.get(i).getName();
            LOG = i;
            break;
        }
    }
    return check_id;
}
```

- 관리자 지정 번호를 입력하면 관리자모드로 이동
- 회원가입을 해야 로그인 가능



```
@Override
public void actionPerformed(ActionEvent e) {
    keyBoardButtonPress(e);

    // 취소 버튼 눌렀을 때 로그인창으로 이동
    if (e.getSource() == back_btn) {
        MainSystem.frame.setContentPane(new Login_Panel());
        MainSystem.frame.revalidate();
    } else if (e.getSource() == join_btn) {
        // 회원가입 버튼 눌렀을 때
        if (name_tf.getText().equals("") || mobile_tf.getText().equals("") || pw_tf.getText().equals("")
            || confirm_pw_tf.getText().equals("")) {
            JOptionPane.showMessageDialog(null, "모든 필드를 입력하세요.", "", JOptionPane.WARNING_MESSAGE);
            return;
        } else if (!pw_tf.getText().equals(confirm_pw_tf.getText())) {
            JOptionPane.showMessageDialog(null, "비밀번호가 맞지 않습니다.", "", JOptionPane.WARNING_MESSAGE);
            return;
        } else if (!isChecked) {
            JOptionPane.showMessageDialog(null, "개인정보취급방침동의를 체크해주세요.", "", JOptionPane.WARNING_MESSAGE);
            return;
        }

        for (int i = 0; i < FileManager.instance.userManager.size(); i++) {
            if (mobile_tf.getText().equals(FileManager.instance.userManager.get(i).getMobile())) {
                JOptionPane.showMessageDialog(null, "동일한 번호가 있습니다.", "Message", JOptionPane.WARNING_MESSAGE);
                return;
            }
        }
    }
}
```

- '취소' 버튼을 누르면 로그인 화면으로 이동
- 모든 필드가 입력되지 않거나, 동일한 휴대폰 번호가 있으면 회원가입 불가
- 개인정보취급방침 동의를 체크해야 회원가입 가능



```
@Override
public void actionPerformed(ActionEvent e) {
    if (e.getSource() == back_btn) {
        MainSystem.frame.setContentPane(new Login_Panel());
        MainSystem.frame.revalidate();
    } else if (e.getSource() == update_btn) {
        UpdateInfo_Dialog dialog = new UpdateInfo_Dialog(moblie);
    } else {
        for (int i = 0; i < seat_btn.length; i++) {
            for (int n = 0; n < seat_btn[i].length; n++) {
                if (e.getSource() == seat_btn[i][n]) {
                    int temp = Integer.parseInt(seat_btn[i][n].getText());
                    if (FileManager.instance.seatManager.get(temp - 1).isSeatUse()) {
                        JOptionPane.showMessageDialog(null, "사용 중인 좌석입니다.", "Message", JOptionPane.INFORMATION_MESSAGE);
                    } else {
                        MainSystem.frame.setContentPane(new Select_Panel(name, moblie, temp - 1));
                        MainSystem.frame.revalidate();
                    }
                }
            }
        }
    }
}

String seatUseState() {
    String state = "";
    int check = 0;
    for (int i = 0; i < FileManager.instance.seatManager.size(); i++) {
        if (!FileManager.instance.seatManager.get(i).isSeatUse()) {
            check += 1;
        }
    }

    state = check + "/" + FileManager.instance.seatManager.size();
    return state;
}
```

- '뒤로가기' 버튼을 누르면 로그인 화면으로 이동
- '정보수정' 버튼을 누르면 비밀번호 수정을 할 수 있는 창 생성
- 사용 중인 자리는 선택 불가



```
void summaryInfo(String useTime) {
    summary_LB.setVisible(true);

    font = new Font("나눔스퀘어", Font.PLAIN, 20);
    seatNum_LB = new JLabel("이용좌석" + seatNum + "번");
    seatNum_LB.setFont(font);
    seatNum_LB.setForeground(Color.BLACK);
    seatNum_LB.setBackground(ColorInfo.instance.dial_button_color);
    seatNum_LB.setOpaque(true);
    seatNum_LB.setVisible(true);
    seatNum_LB.setBounds(70, 740, 240, 40);
    add(seatNum_LB);

    useTime_LB = new JLabel("이용시간" + useTime);
    useTime_LB.setFont(font);
    useTime_LB.setForeground(Color.BLACK);
    useTime_LB.setBackground(ColorInfo.instance.dial_button_color);
    useTime_LB.setOpaque(true);
    useTime_LB.setVisible(true);
    useTime_LB.setBounds(70, 780, 240, 40);
    add(useTime_LB);
}

} else if (e.getSource() == hourTicket_rd) {
    hour_LB.setVisible(true);
    radio_visible(hour_rd);
    radio_invisible(fixed_rd);
    repaint();
} else if (e.getSource() == fixedTicket_rd) {
    hour_LB.setVisible(true);
    radio_visible(fixed_rd);
    radio_invisible(hour_rd);
    repaint();
} else if (hourTicket_rd.isSelected()) {
    for (int i = 0; i < hour_rd.length; i++) {
        if (e.getSource() == hour_rd[i]) {
            summary_invisible();
            useTime = hour_rd[i].getText();
            summaryInfo(useTime);
            repaint();
        }
    }
} else if (fixedTicket_rd.isSelected()) {
    for (int i = 0; i < fixed_rd.length; i++) {
        if (e.getSource() == fixed_rd[i]) {
            summary_invisible();
            useTime = fixed_rd[i].getText();
            summaryInfo(useTime);
            repaint();
        }
    }
}
}
```

- 선택한 상품에 따라 시간이 다르게 표시
- 상품과 시간을 모두 선택하면 화면에 내역 확인 출력

```

void checkIn(String mobile, String updateTime, int seatNum) {
    LOG = -1;
    for (int i = 0; i < userManager.size(); i++) {
        if (mobile.equals(userManager.get(i).getMobile())) {
            userManager.get(i).setSeatNum(seatNum);
            userManager.get(i).setMaxTime(Integer.parseInt(updateTime));
            userManager.get(i).setPreTime(format(System.currentTimeMillis()));
            userManager.get(i).seatUse = true;
            updateUser(i, userManager.get(i));
            LOG = i;
            break;
        }
    }

    if (LOG == -1) {
        JOptionPane.showMessageDialog(null, "비정상적인 접근입니다.", "Message", JOptionPane.WARNING_MESSAGE);
        System.exit(0);
    }

    for (int i = 0; i < seatManager.size(); i++) {
        if (seatNum == seatManager.get(i).getSeatNum()) {
            seatManager.get(i).setSeatUse(true);
            saveSeatData();
        }
    }

    JOptionPane.showMessageDialog(null, "영수증이 출력되었습니다.", "Message", JOptionPane.PLAIN_MESSAGE);
    MainSystem.frame.setContentPane(new Login_Panel());
    MainSystem.frame.revalidate();
}

```

- 잘못된 접근일 경우 시스템이 종료
- 카드 결제 클릭 시 '준비 중' 메시지 알림
- 결제가 완료되면 로그인 화면으로 이동



```
public void userListSet() {
    Component[] componentList = this.getComponents();

    for (Component c : componentList) {
        if (c instanceof UserInfo_Board) {
            this.remove(c);
        }
    }

    int start_index = (page_num - 1) * page_user_max;
    int total = user_list.size() - start_index;
    int count = page_user_max;

    if (total < page_user_max) {
        count = total;
    }

    user_board_list = new ArrayList<>();
    for (int i = 0; i < count; i++) {
        UserInfo_Board userBoard = new UserInfo_Board(i + start_index, user_list.get(start_index + i), this);
        add(userBoard); // 컨포넌트에 추가
        user_board_list.add(userBoard);
    }

    @Override
    public void actionPerformed(ActionEvent e) {
        if (e.getSource() == checkOut_btn && user.getSeatNum() != 0) {
            int result = JOptionPane.showConfirmDialog(null, user.getName() + "님을 강제퇴실 하시겠습니까?", "Message",
                JOptionPane.YES_NO_OPTION);
            if (result == JOptionPane.YES_OPTION) {
                if (FileManager.instance checkOut(user.getMobile())) {
                    JOptionPane.showMessageDialog(null, "강제퇴실이 완료되었습니다.", "Message", JOptionPane.INFORMATION_MESSAGE);
                    MainSystem.frame.setContentPane(new Admin_UserInfo_Panel());
                    MainSystem.frame.revalidate();
                }
            }
        }
    }

    repaint();
}
```

- 회원 가입한 사용자 정보를 게시판 형식으로 출력
- '강제퇴실' 버튼을 클릭하면 자리가 있는 회원은 강제 퇴실



```
String endTime(UserInfo user) {
    String endTime = "";
    if (!user.getPreTime().equals("0")) {
        Date date = null;
        try {
            date = FileManager.instance.format.parse(user.getPreTime());
        } catch (ParseException e) {
            e.printStackTrace();
        }
        Calendar cal = Calendar.getInstance();
        cal.setTime(date);
        cal.add(Calendar.HOUR, user.getMaxTime());
        endTime = FileManager.instance.format.format(cal.getTime());
    }
    return endTime;
}
```

- 사용 중인 좌석을 클릭하면 해당 좌석을 이용하는 회원의 정보가 출력

```
void seatStateInfo(UserInfo user) {
    font = new Font("나눔스퀘어", Font.PLAIN, 16);
    infoName_LB = new JLabel("이름 : " + user.getName());
    infoName_LB.setFont(font);
    infoName_LB.setForeground(Color.white);
    infoName_LB.setBounds(40, 80, 200, 20);
    infoName_LB.setVisible(true);
    add(infoName_LB);

    infoMobile_LB = new JLabel("H.P : " + user.getMobile());
    infoMobile_LB.setFont(font);
    infoMobile_LB.setForeground(Color.white);
    infoMobile_LB.setBounds(40, 100, 200, 20);
    infoMobile_LB.setVisible(true);
    add(infoMobile_LB);

    infoPreTime_LB = new JLabel("시작 시간 : " + user.getPreTime());
    infoPreTime_LB.setFont(font);
    infoPreTime_LB.setForeground(Color.white);
    infoPreTime_LB.setBounds(40, 120, 280, 20);
    infoPreTime_LB.setVisible(true);
    add(infoPreTime_LB);

    infoMaxTime_LB = new JLabel("종료 시간 : " + endTime(user));
    infoMaxTime_LB.setFont(font);
    infoMaxTime_LB.setForeground(Color.white);
    infoMaxTime_LB.setBounds(40, 140, 280, 20);
    infoMaxTime_LB.setVisible(true);
    add(infoMaxTime_LB);
}
```

#04

2021

JSP MVC2 구조 정리

마켓컬리 관리자

마켓컬리 관리자

JSP MVC2 구조 정리

관리자 기능 구조를 공부함으로써 웹 애플리케이션의 전체적인 흐름과 구성을 파악하는 프로젝트입니다.

개요

- 소개 : 온라인 식재료 판매 업체 관리자 페이지
- 선정 이유 : 식재료 상품 목록을 게시판으로 관리 가능
쇼핑몰 홈페이지의 전체적인 구성 파악 가능

개발 환경

- Build Tool : Eclipse
- Server : apache-tomcat-8.5.37
- DB : MYSQL
- Computer Language : HTML, CSS, JS, JSP, JAVA Servlet



1

상품 관련

1. 상품 등록 : 새로운 상품 등록
2. 상품 수정/삭제 : 기존 상품 수정 및 삭제

2

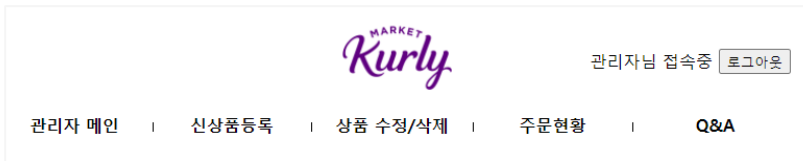
구매된 상품 관련

1. 전체 구매 목록 확인 : 전체 상품의 목록 확인 및 관리
2. 주문 목록 확인 : 주문 목록 확인 및 관리

3

Q&A 관련

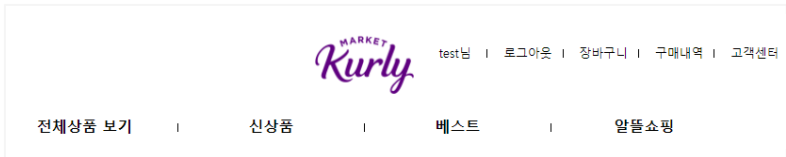
1. 전체 게시물 보기 : 전체 게시물 목록 출력
2. 게시물 보기 : 게시물 출력
3. 답변글 작성 : 답변글 등록



1

회원 관련

1. 회원가입: 아이디 체크, 이메일 체크 필수
2. 로그인: 로그인해야 상품 주문 가능
3. 장바구니: 선택한 상품 장바구니에 추가/삭제
4. 구매내역: 주문내역 출력



2

상품 관련

1. 전체 상품 보기: 전체 상품 목록 출력
2. 신상품 보기: 신상품 목록 출력
3. 베스트 상품 보기: 베스트 상품 목록 출력
4. 알뜰 쇼핑 상품 보기: 알뜰 쇼핑 상품 목록 출력

3

고객센터 관련

1. 게시글 작성: 새로운 게시글 등록
2. 전체 게시글 보기: 전체 게시글 목록 출력
3. 게시글 보기: 게시글 출력

테이블명	customer (회원)		
NO	Name	Field	Type
1	아이디	id	VARCHAR(50)
2	비밀번호	passwd	VARCHAR(16)
3	이름	name	VARCHAR(10)
4	가입날짜	reg_date	VARCHAR(50)
5	전화번호	tel	VARCHAR(20)
6	주소	address	VARCHAR(100)
7	이메일	email	VARCHAR(20)
테이블명	manager (관리자)		
NO	Name	Field	Type
1	관리자 아이디	id	VARCHAR(50)
2	관리자 비밀번호	pw	VARCHAR(50)
3	관리자 이름	name	VARCHAR(50)

테이블명	board (게시판)		
NO	Name	Field	Type
1	고유번호	num	INT(10)
2	작성자	writer	VARCHAR(20)
3	제목	title	VARCHAR(50)
4	비밀번호	pw	VARCHAR(20)
5	작성일	reg_date	DATE
6	그룹화 아이디	ref	INT(10)
7	글 순서	re_step	INT(10)
8	글 레벨	re_level	INT(10)
9	조회수	readcount	INT(10)
10	내용	content	VARCHAR(100)

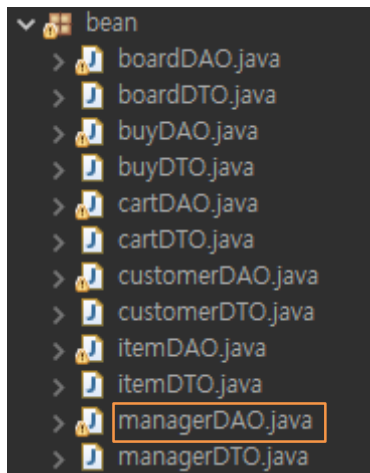
테이블명	Item (상품)		
NO	Name	Field	Type
1	상품 번호	item_number	INT(50)
2	상품 종류	item_category	VARCHAR(3)
3	상품 이름	item_name	VARCHAR(16)
4	상품 가격	item_price	INT(50)
5	상품 재고	item_stock	INT(50)
6	상품 이미지	item_image	VARCHAR(10)
7	상품 정보	item_info	VARCHAR(100)
8	할인율	discount_rate	INT(50)
9	등록 날짜	reg_date	VARCHAR(50)
10	매진 유무	sold	INT(50)

테이블명	cart (장바구니)		
NO	Name	Field	Type
1	번호	cart_number	INT(50)
2	구매자	buyer	VARCHAR(50)
3	상품 이름	item_name	VARCHAR(16)
4	구매 가격	buy_price	INT(50)
5	구매 개수	buy_count	INT(50)
6	상품 이미지	item_image	VARCHAR(50)

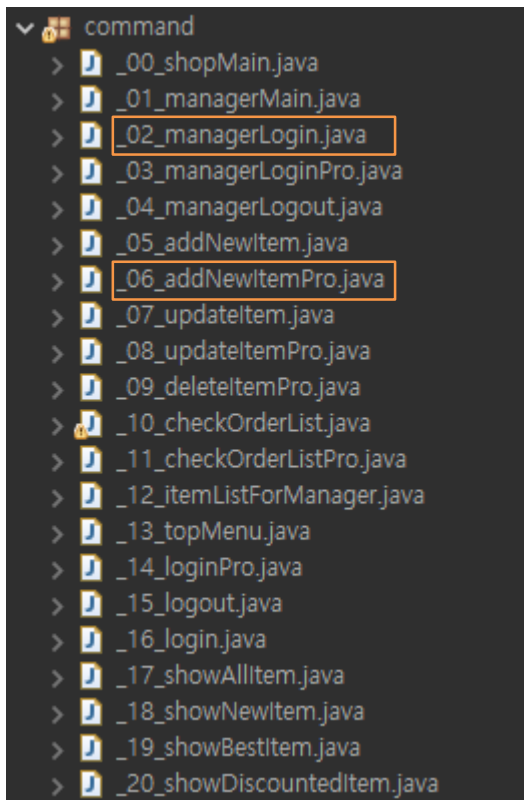
테이블명	buy (주문)		
NO	Name	Field	Type
1	구매자 아이디	customer_id	VARCHAR(50)
2	구매자 이름	customer_name	VARCHAR(50)
3	장바구니 번호	cart_number	INT(50)
4	상품 이름	item_name	VARCHAR(50)
5	구매 가격	buy_price	INT(50)
6	구매 개수	buy_count	INT(50)
7	상품 이미지	item_image	VARCHAR(50)
8	구매 날짜	buy_date	VARCHAR(50)
9	지불 방법	howpay	VARCHAR(50)
10	주소	address	VARCHAR(100)

Model
Command
Controller

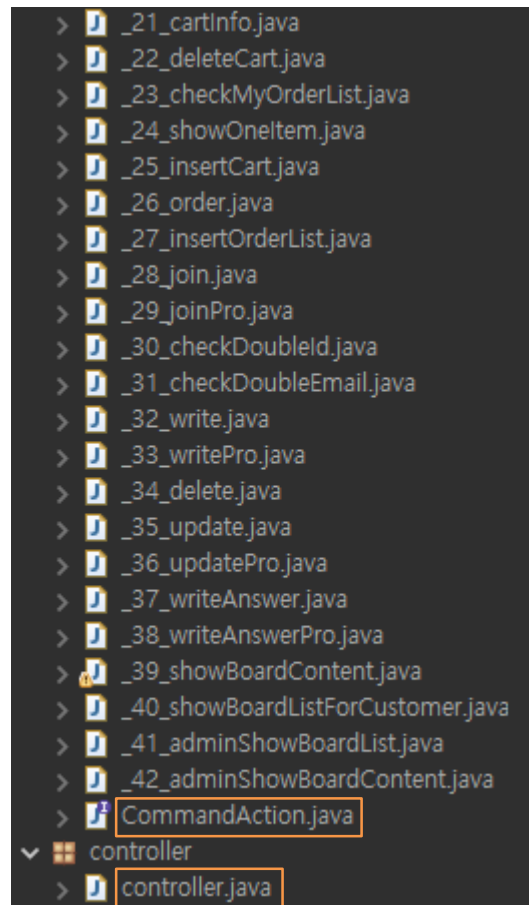
JAVA



JAVA SERVLET



- 사용자의 모든 요청을 Controller로 전달
- Controller에서 분기된 명령을 Command에서 직접 처리



```
public class managerDAO {

    public static managerDAO instance = new managerDAO();

    Connection conn = null;
    PreparedStatement pstmt = null;
    ResultSet rs = null;

    private Connection getConnection() throws Exception {
        Context initCtx = new InitialContext();
        Context envCtx = (Context) initCtx.lookup("java:comp/env");
        DataSource ds = (DataSource) envCtx.lookup("jdbc/pool");
        conn = ds.getConnection();
        return conn;
    }
}
```

```
public int checkManager(String id, String pw) {
    int check=-1;
    try {
        conn = getConnection();
        String sql = "select * from manager where id=? and pw=?";
        pstmt = conn.prepareStatement(sql);
        pstmt.setString(1, id);
        pstmt.setString(2, pw);
        rs=pstmt.executeQuery();

        if(rs.next()) {
            check=1;
        }
    }catch(Exception e) {
        e.printStackTrace();
    }finally {
        if(conn!=null)
            try {conn.close();}catch(SQLException sqle) {}
        if(pstmt!=null)
            try {pstmt.close();}catch(SQLException sqle) {}
        if(rs!=null)
            try {rs.close();}catch(SQLException sqle) {}
    }
    return check;
}
```

```
public void insertNewItem(itemDTO dto) {
    try {
        conn = getConnection();

        int num=0;
        String sql = "select max(item_number) from item";
        pstmt = conn.prepareStatement(sql);
        rs = pstmt.executeQuery();

        if(rs.next()) {
            num = rs.getInt(1);
        }

        sql = "insert into item(item_number, item_category,item_name,item_price,item_stock,item_image,"
            + "item_info.discount_rate,reg_date,sold) values(?, ?, ?, ?, ?, ?, ?, ?, ?, ?)";
        pstmt = conn.prepareStatement(sql);
        pstmt.setInt(1, num+1);
        pstmt.setString(2, dto.getItem_category());
        pstmt.setString(3, dto.getItem_name());
        pstmt.setInt(4, dto.getItem_price());
        pstmt.setInt(5, dto.getItem_stock());
        pstmt.setString(6, dto.getItem_image());
        pstmt.setString(7, dto.getItem_info());
        pstmt.setInt(8, dto.getDiscount_rate());

        pstmt.executeUpdate();
    }catch(Exception e) {
        e.printStackTrace();
    }finally {
        if(conn!=null)
            try {conn.close();}catch(SQLException sqle) {}
        if(pstmt!=null)
            try {pstmt.close();}catch(SQLException sqle) {}
        if(rs!=null)
            try {rs.close();}catch(SQLException sqle) {}
    }
}
```

- DB 연동 메서드
- 관리자 인증 메서드
- 관리자 정보 반환 메서드
- 상품 삭제 메서드
- 상품 등록 메서드
- 상품 수정 메서드
- 상품 목록 반환 메서드
- 주문 목록 반환 메서드
- 회원 수 반환 메서드
- 상품 개수 반환 메서드
- 주문 수 반환 메서드

```
import javax.servlet.http.HttpServletRequest;

public interface CommandAction {
    public String requestPro(
        HttpServletRequest request, HttpServletResponse response) throws Throwable;
}
```

- 각 Servlet에서 상속을 받도록 구현된 인터페이스
- 요청 파라미터로 명령어를 전달하는 방식의 슈퍼 인터페이스

```
public class _03_managerLoginPro implements CommandAction{

    @Override
    public String requestPro(HttpServletRequest request, HttpServletResponse response) throws Throwable {
        String id = request.getParameter("id");
        String pw = request.getParameter("pw");

        int check = managerDAO.instance.checkManager(id, pw);

        request.setAttribute("check", new Integer(check));
        request.setAttribute("id", id);
        return "/03_managerLoginPro.jsp";
    }
}
```

- CommandAction 인터페이스 구현
- 가져온 파라미터 값으로 관리자 인증
- setAttribute() 메서드로 속성 값을 저장
- 요청이 처리되면 view의 경로와 세션 ID를 반환

addItemPro

```
public class _06_addNewItemPro implements CommandAction{

    @Override
    public String requestPro(HttpServletRequest request, HttpServletResponse response) throws Throwable {
        request.setCharacterEncoding("UTF-8");

        String realFolder="";
        String filename="";
        MultipartRequest imageup = null;

        String saveFolder="/img";
        String encType = "utf-8";
        int max_size=2 * 1024 * 1024;

        ServletContext context = request.getSession().getServletContext();
        realFolder = context.getRealPath(saveFolder);

        try{
            imageup = new MultipartRequest(request, realFolder, max_size,encType, new DefaultFileRenamePolicy());
            Enumeration<?> files = imageup.getFileNames();

            while(files.hasMoreElements()){
                String name =(String)files.nextElement();
                filename = imageup.getFileName(name);
            }

        }catch(Exception e){
            e.printStackTrace();
        }
    }
}
```

- 상품 등록 시 이미지 파일 업로드를 위한 MultipartRequest 객체 생성
- 한 번에 올릴 수 있는 파일 용량 2MB로 제한
- 서버의 물리 경로를 구하여 업로드 디렉토리 설정

controller

```

@WebServlet(urlPatterns = {"*.do"})
public class controller extends HttpServlet {

    private static final long serialVersionUID = 1L;
    private Map<String, Object> commandMap = new HashMap<String, Object>();

    @Override
    public void init() throws ServletException {
        commandMap.put("/index.do", new _00_shopMain());
        commandMap.put("/managerMain.do", new _01_managerMain());
        commandMap.put("/managerLogin.do", new _02_managerLogin());
        commandMap.put("/managerLoginPro.do", new _03_managerLoginPro());
        commandMap.put("/managerLogout.do", new _04_managerLogout());
        commandMap.put("/addNewItem.do", new _05_addNewItem());
        commandMap.put("/addNewItemPro.do", new _06_addNewItemPro());
        commandMap.put("/updateItem.do", new _07_updateItem());
        commandMap.put("/updateItemPro.do", new _08_updateItemPro());
        commandMap.put("/deleteItemPro.do", new _09_deleteItemPro());
        commandMap.put("/checkOrderList.do", new _10_checkOrderList());
        commandMap.put("/checkOrderListPro.do", new _11_checkOrderListPro());
        commandMap.put("/itemListForManager.do", new _12_itemListForManager());
    }
}

```

- 서블릿 초기화 시 무조건 실행되는 init() 메서드 오버라이딩
- Map 인터페이스를 구현한 HashMap 객체로 key-value 저장

- 사용자의 요청을 분석해서 해당 작업을 처리하는 메서드
- view에서 넘어온 프로젝트 path로 해당 Command의 requestPro() 메서드 호출
- 반환 받은 view로 페이지 이동

```

protected void requestPro(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    String view = null;
    CommandAction com=null;
    try {
        String command = request.getRequestURI();
        if(command.indexOf(request.getContextPath()) == 0)
            command = command.substring(request.getContextPath().length());
        com = (CommandAction)commandMap.get(command);
        view = com.requestPro(request, response);
        System.out.println("com = " + com);
        System.out.println("view = " + view);
    } catch (Throwable e) {
        e.printStackTrace();
    }
    request.setAttribute("cont", view);
    RequestDispatcher dispatcher =
        request.getRequestDispatcher("/00_index.jsp");
    dispatcher.forward(request, response);
}

protected void doGet(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    requestPro(request, response);
}

protected void doPost(HttpServletRequest request, HttpServletResponse response) throws ServletException, IOException {
    requestPro(request, response);
}
}

```


View

- 00_index.jsp
- 00_shopMain.jsp
- 01_managerMain.jsp
- 02_managerLogin.js
- 02_managerLogin.jsp
- 03_managerLoginPro.jsp
- 04_managerLogout.jsp
- 05_addNewItem.js
- 05_addNewItem.jsp
- 06_addNewItemPro.jsp
- 07_updateItem.jsp
- 08_updateItemPro.jsp
- 09_deleteItemPro.jsp
- 10_checkOrderList.jsp
- 11_checkOrderListPro.jsp
- 12_itemListForManager.js
- 12_itemListForManager.jsp
- 13_topMenu.jsp
- 14_loginPro.jsp
- 15_logout.jsp
- 16_login.jsp
- 17_showAllItem.jsp
- 18_showNewItem.jsp
- 19_showBestItem.jsp
- 20_showDiscountedItem.jsp

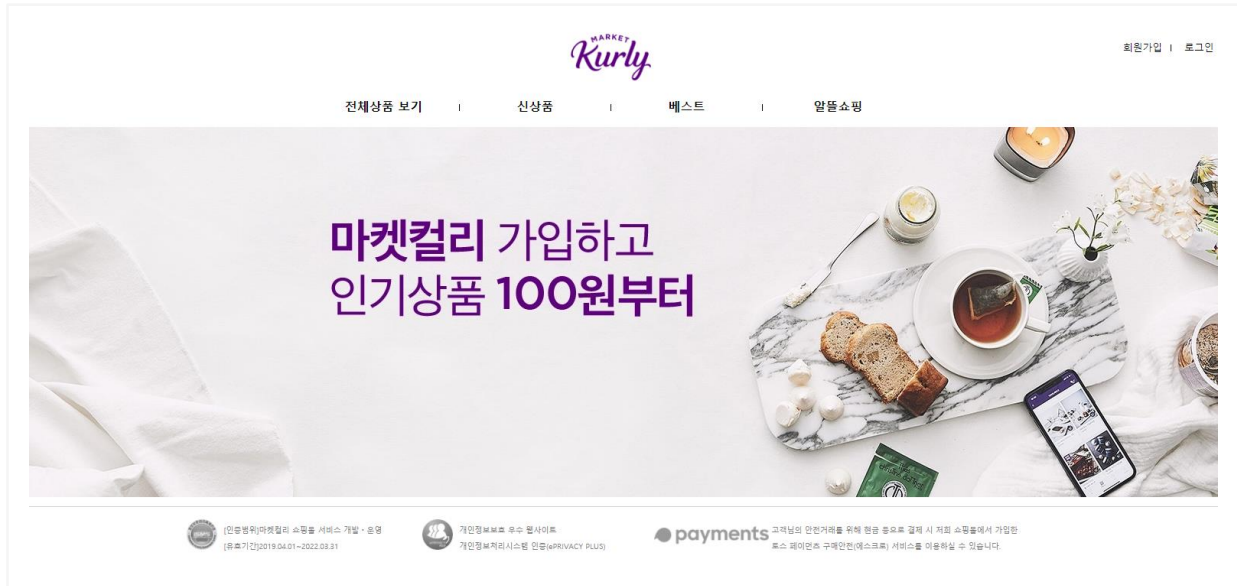
- 21_cartInfo.jsp
- 22_deleteCart.jsp
- 23_checkMyOrderList.jsp
- 24_showOneItem.jsp
- 25_insertCart.jsp
- 26_order.jsp
- 27_insertOrderList.jsp
- 28_join.js
- 28_join.jsp
- 29_joinPro.jsp
- 30_checkDoubleId.jsp
- 31_checkDoubleEmail.jsp
- 32_write.jsp
- 33_writePro.jsp
- 34_delete.jsp
- 35_update.jsp
- 36_updatePro.jsp
- 37_writeAnswer.jsp
- 38_writeAnswerPro.jsp
- 39_showBoardContent.jsp
- 40_showBoardListForCustomer.jsp
- 41_adminShowBoardList.jsp
- 42_adminShowBoardContent.jsp

- 클라이언트의 요청에 대한 응답을 UI 화면으로 출력
- 링크는 *.do로 잡아서 Controller로 이동
- 관리자 로그인, 상품 목록, 상품 등록, 회원가입 Ajax 사용

JSP & JAVASCRIPT

```
<div id="content" class="box2">
  <jsp:include page="${cont}" />
</div>
```

- <jsp:include> 액션 태그 이용
- Controller에서 받아온 페이지를 포함하여 동적인 페이지 구성



managerLogin.js
managerLogin.jsp

```
$(document).ready(function(){
    $("#login").click(function(){
        var query = {id:$("#id").val(),
                    pw:$("#pw").val()};

        $.ajax({
            type: "POST",
            url: "/MVC_marketKurly/managerLoginPro.do",
            data: query,
            success: function(data){
                window.location.href="/MVC_marketKurly/managerMain.do";
            }
        });
    });
});
```

- 로그인 버튼을 클릭하면 함수 실행
- 전송할 데이터를 해당 Controller로 보내고 성공하면 이동할 Controller 호출

```
<script src="/MVC_marketKurly/02_managerLogin.js"></script>

<c:if test="${empty sessionScope.id}">
    <div id="status">
        <ul>
            <li>
                <label for="id">아이디</label>
                <input id="id" name="id" type="email" size="20" maxlength="50" placeholder="marketkurly">
                <label for="pw">비밀번호</label>
                <input id="pw" name="pw" type="password" size="20" placeholder="6~16자 숫자/문자" maxlength="16">
                <button id="login">로그인</button>
            </li>
        </ul>
    </div>
</c:if>
```



아이디 비밀번호

addNewItem.js
addNewItem.jsp

```
<script src="/MVC_marketKurly/05_addNewItem.js"></script>
<c:if test='${empty sessionScope.id}'>
  <meta http-equiv="Refresh" content="0;url=/MVC_marketKurly/managerMain.do">
</c:if>
```

```
$(document).ready(function(){
    $("#upForm1").ajaxForm({
        success: function(data, status){
            window.location.href="/MVC_marketKurly/itemListForManager.do";
        }
    });
});
```

```
<form method="post" id="upForm1" action="/MVC_marketKurly/addNewItemPro.do" enctype="multipart/form-data" accept-charset="UTF-8">
  <table border="1" style="border-collapse: collapse;">
    <tr height="30">
      <td width="200">상품 카테고리</td>
      <td width="500">
        <select name="category">
          <option value="100">채소</option>
          <option value="200">해산물</option>
          <option value="300">육류</option>
          <option value="400">전자제품</option>
        </select>
      </td>
    </tr>
  </table>
```

```
<tr height="30">
  <td width="200">이미지</td>
  <td width="500">
    <input type="file" name="image">
  </td>
</tr>
<tr height="30">
  <td width="200">상품정보</td>
  <td width="500">
    <input type="text" name="info" size="50">
  </td>
</tr>
<tr height="30">
  <td width="200">할인율</td>
  <td width="500">
    <input type="text" name="discountRate" value="">
  </td>
</tr>
</table>
<input type="submit" id="registItem" value="상품 추가하기">
</form>
```

- 관리자가 아니면 메인 페이지로 이동
- 상품 추가가 성공하면 상품목록 페이지로 이동
- 서버로 파일을 전송 시 enctype은 "Multipart/form-data" 로 지정
- method 속성은 반드시 post 타입으로 지정


관리자님 접속중 [로그아웃](#)

[관리자 메인](#) |
 [신상품등록](#) |
 [상품 수정/삭제](#) |
 [주문현황](#) |
 [Q&A](#)

상품 카테고리	채소 ▼
상품명	
상품가격	원
상품재고	
이미지	파일 선택 선택한 파일 없음
상품정보	
할인율	%
상품 추가하기	

itemListForManager.js
itemListForManager.jsp

```
<script src="/MVC_marketKurlly/12_itemListForManager.js"></script>
<:if test= "${empty sessionScope.id}" >
<meta http-equiv="Refresh" content="0:url=/MVC_marketKurlly/managerMain.do">
</:if>
```

```
function edit(editBtn){
    var str = editBtn.name;
    var query = "/MVC_marketKurlly/updateItem.do?item_number="+str;
    window.location.href=query;
}
function del(delBtn){
    var str = delBtn.name;
    var query = "/MVC_marketKurlly/deleteItemPro.do?item_number="+str;
    window.location.href=query;
}
```

- 관리자가 아니면 메인 페이지로 이동
- Controller에서 가져온 상품 목록을 출력
- 수정/삭제 버튼을 누르면 GET 방식으로 데이터를 해당 Controller로 전송하여 명령 처리

```
<:set var="number" value= "${0}" >
<:forEach var = "item" items= ${registeredItemList} >
|  |  |  |  |  |  |  |  |  |  |  |  |
| --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- | --- |
| <:set var="number" value= ${number+1} > <:out value= ${number} > </td>  <:if test= "${item.getItem_category()=='100'}">채소</:if> <:if test= "${item.getItem_category()=='200'}">해산물</:if> <:if test= "${item.getItem_category()=='300'}">육류</:if> <:if test= "${item.getItem_category()=='400'}">전지제품</:if> </td>  ${item.getItem_name()} </td>  ${item.getItem_price()} </td>  ${item.getItem_stock()} </td>  <img src = "/MVC_marketKurlly/img_ ${item.getItem_image()}" height="50"> </td>  ${item.getItem_info()} </td>  ${item.getDiscount_rate()} </td>  ${item.getReg_date()} </td>  ${item.getSold()} </td>  <input type="hidden" id="item_number" name="item_number" value= ${item.getItem_number()} > <button id="edit" name= ${item.getItem_number()} "onclick="edit(this)">수정</button> </td>  <button id="del" name= ${item.getItem_number()} "onclick="del(this)">삭제</button> </td> </tr> </forEach> | | | | | | | | | | | |

```



관리자님 접속중

관리자 메인 | 신상품등록 | 상품 수정/삭제 | 주문현황 | Q&A

번호	카테고리	상품명	가격	재고	이미지	상품정보	할인율	등록일	판매량	수정/삭제
1	채소	대파	1200	3		대파입니다	0	2021-02-22 01:49:25	11	수정 삭제
2	채소	상추	2000	4		상추입니다	10	2021-02-22 01:49:25	0	수정 삭제
3	채소	당근	2500	8		당근입니다	10	2021-02-22 01:49:25	0	수정 삭제

#05

2021

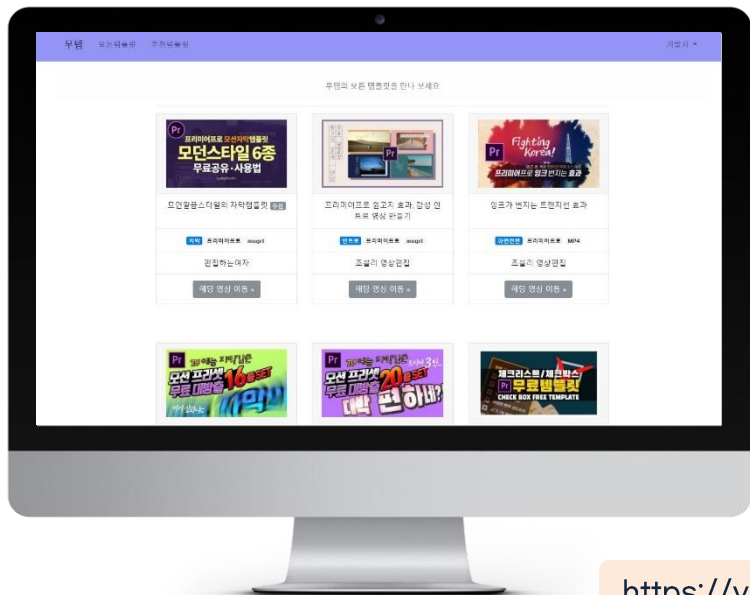
Spring Framework 프로젝트

MUTEM

무템 (mutem)

무료 영상 템플릿 사이트

무템은 유튜버들이 제공하는 영상용 무료 템플릿을 정리한 사이트입니다.



<https://youtu.be/JKkO0zufpml>

<https://github.com/seoheejung/mutem/tree/master/mutem>

개요

- 스프링 프레임워크 기반 반응형 웹 사이트 프로젝트
- 영상 제작을 위한 템플릿 찾는 사람들을 위한 서비스

개발 인원 및 기간

- 개발 기간 : 2021.07
- 개발 인원 : 1명

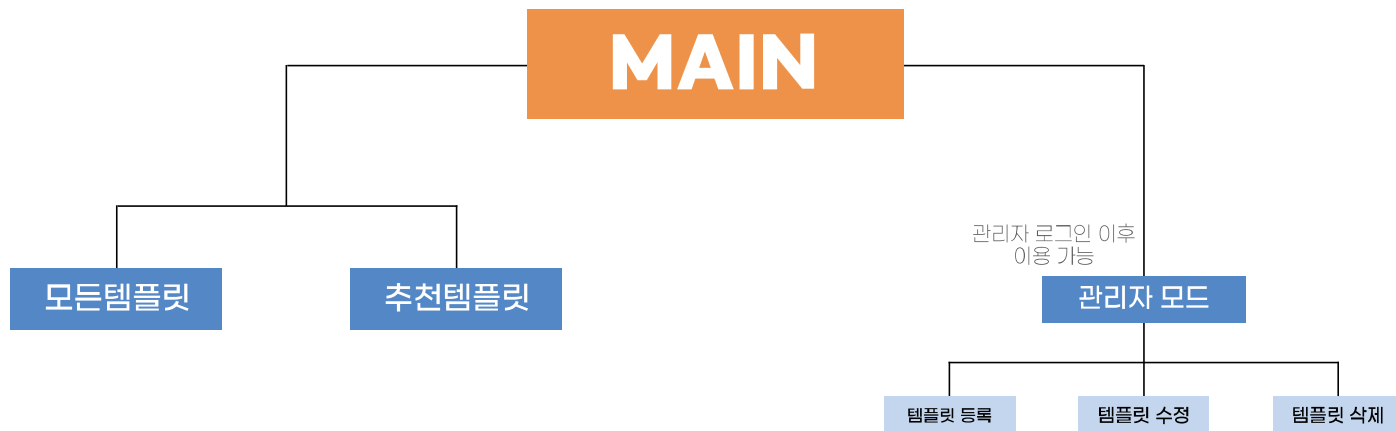
적용 기술

- Spring FrameWork
- JSP / JAVA Servlet
- MYSQL
- Bootstrap (HTML/CSS/JavaScript/JQuary)

기여도

- 기획
- 개발
- 디자인





1

화면 관련

1. 모든 템플릿 보기 : 모든 템플릿 목록 출력
2. 추천 템플릿 보기 : 추천이 된 템플릿 목록만 출력

2

관리자모드 관련

1. 로그인 : 관리자모드 진입을 위한 로그인 필수
2. 템플릿 관리 : 관리자 로그인을 하지 않으면 접근 불가

localhost:8090 내용:

잘못된 경로입니다. 관리자모드는 로그인이 필요합니다.

확인

3

템플릿 관련

1. 템플릿 등록 : 새로운 템플릿 등록
2. 템플릿 수정/삭제 : 기존 템플릿 수정 및 삭제

테이블명	template (템플릿)		
NO	Name	Field	Type
1	등록번호 (PK)	tem_number	INT (AUTO_INCREMENT)
2	카테고리	tem_category	VARCHAR(20)
3	템플릿 이름	tem_name	VARCHAR(300)
4	유튜브 채널	tem_channel	VARCHAR(100)
5	유튜브 링크	tem_link	VARCHAR(100)
6	유튜브 동영상 코드	tem_linkshare	VARCHAR(50)
7	파일 형식	tem_format	VARCHAR(50)
8	프로그램	tem_program	VARCHAR(100)
9	등록날짜	reg_date	TIMESTAMP
10	추천	recommendation	INT (DEFAULT 0)

테이블명	admin (관리자)		
NO	Name	Field	Type
1	관리자 아이디	id	VARCHAR(10)
2	관리자 비밀번호	pw	VARCHAR(10)
3	관리자 이름	name	VARCHAR(10)

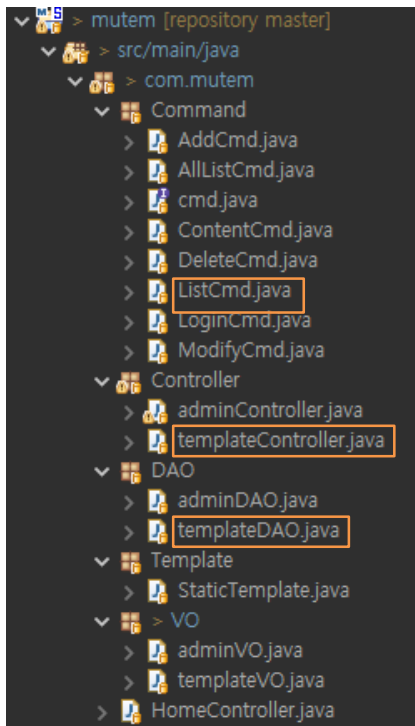
코드 목록

src/main/java
src/main/webapp

Spring mvc 흐름

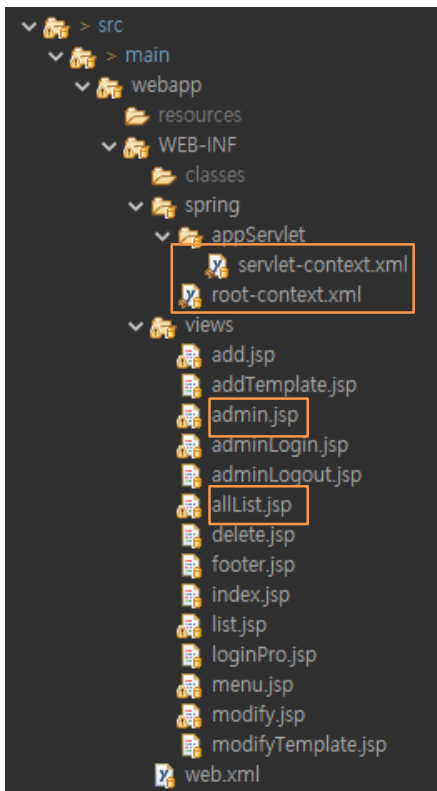
- 1) Client -> Dispatcher Servlet
클라이언트가 Url로 접근하여 정보를 요청
- 2) Dispatcher Servlet -> Handler Mapping
Handler Mapping에게 매핑된 컨트롤러 요청
- 3) Handler Mapping -> Controller
Controller에게 요청 전송
- 4) Controller -> Dispatcher Servlet
요청의 결과와 view에 대한 정보 반환
- 5) Dispatcher Servlet -> ViewResolver
응답할 view를 요청
- 6) ViewResolver -> View
- 7) View -> Dispatcher Servlet
처리결과가 포함된 View 전달
- 8) Dispatcher Servlet -> Client
결과 반환

JAVA



- Command : 데이터를 가져오는 Service 역할
- Controller : Dispatcher Servlet(servlet-context.xml)로 받았을 때 검사용
- DAO : 데이터베이스에 접근하기 위해 만든 객체
- VO : 값을 저장하기 위한 객체 (DTO)
- Template : 싱글톤 패턴을 사용하여 JdbcTemplate 인스턴스를 1개만 생성하기 위해 선언

webapp



root-context.xml
servlet-context.xml

```
<!-- 해당 뷰로 넘어가기 위해 매핑하기 -->
<bean name="/index" class="com.mutem.Controller.templateController" />
<bean name="/list" class="com.mutem.Controller.templateController" />
<bean name="/allList" class="com.mutem.Controller.templateController" />

<bean name="/admin" class="com.mutem.Controller.adminController" />
<bean name="/adminLogin" class="com.mutem.Controller.adminController" />
<bean name="/login" class="com.mutem.Controller.adminController" />
<bean name="/adminLogout" class="com.mutem.Controller.adminController" />

<bean name="/add" class="org.springframework.web.servlet.view.InternalResourceViewResolver">
  <beans:property name="prefix" value="/WEB-INF/views/" />
  <beans:property name="suffix" value=".jsp" />
</beans:bean>

<!-- 빈으로 등록 될 준비를 마친 클래스들을 스캔하여, 빈으로 등록하기 -->
<context:component-scan base-package="com.mutem" />

<!-- DataSource 설정, spring-jdbc 모듈 사용, spring 에서 jdbc 를 통해 mysql 에 접속하기-->
<beans:bean name="dataSource"
  class="org.springframework.jdbc.datasource.DriverManagerDataSource">
  <beans:property name="driverClassName" value="com.mysql.cj.jdbc.Driver" />
  <beans:property name="url" value="jdbc:mysql://127.0.0.1:3306/mutem" />
  <beans:property name="username" value="root" />
  <beans:property name="password" value="root" />
</beans:bean>

<!-- JdbcTemplate 설정 -->
<beans:bean name="template"
  class="org.springframework.jdbc.core.JdbcTemplate">
  <beans:property name="dataSource" ref="dataSource" />
</beans:bean>
```

- DataSource 설정, JdbcTemplate 설정, View페이지 경로 및 파일명 지정 등을 위한 DispatcherServlet

```
@Controller
public class templateController {

    cmd cmd = null;

    private JdbcTemplate template;

    @Autowired
    public void setTemplate(JdbcTemplate template){
        this.template = template;
        StaticTemplate.template = this.template;
    }

    @RequestMapping("/index")
    public String index(Model model){
        cmd = new ListCmd();
        cmd.service(model);

        return "index";
    }
}
```

- @RequestMapping 어노테이션을 이용하여 index 요청 매핑
- View에 데이터를 전달하기 위한 Model을 인자로 넘겨서 서비스를 실행
- Client -> Dispatcher Servlet -> Controller -> Service 클래스 순으로 요청

```
public class ListCmd implements cmd{
    @Override
    public void service(Model model) {
        templateDAO dao = new templateDAO();
        ArrayList<templateVO> tVO = dao.list();
        model.addAttribute("list", tVO);
    }
}
```

- 게시판의 리스트를 DB로부터 가져오는 객체
- list 변수에 tVO를 추가하여 반환

```

public class templateDAO {
    JdbcTemplate template;

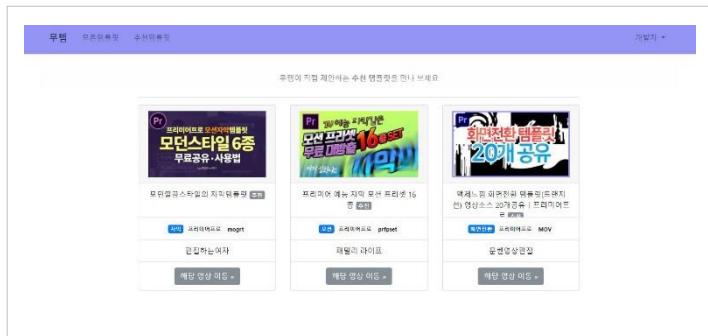
    // 생성자
    public templateDAO() {
        // 생성자에서 StaticTemplate 패키지의 static 프로퍼티(변수)의 template를 불러온다.
        this.template = StaticTemplate.template;
    };

    // 리스트 반환 메서드
    public ArrayList<templateVO> list() {
        ArrayList<templateVO> tVO = null;
        String sql = "select tem_number, tem_category, tem_name, tem_channel, tem_link, tem_linkshare,"
            + " tem_format, tem_program, reg_date, recommendation"
            + " from template where recommendation = 1 "
            + " order by tem_number asc";

        RowMapper<templateVO> rm = new BeanPropertyRowMapper<templateVO>(<templateVO.class>);
        tVO = (ArrayList<templateVO>) template.query(sql, rm);
        return tVO;
    }
}

```

- DAO 객체 생성 시 JdbcTemplate을 선언 없이 사용하기 위해 생성자에서 호출
- 템플릿 데이터베이스에 접근하여 추천이 된 템플릿 리스트를 반환
- RowMapper : 쿼리를 처리하는 객체
- BeanPropertyRowMapper : Bean 객체를 기반으로 RowMapper를 생성하는 객체



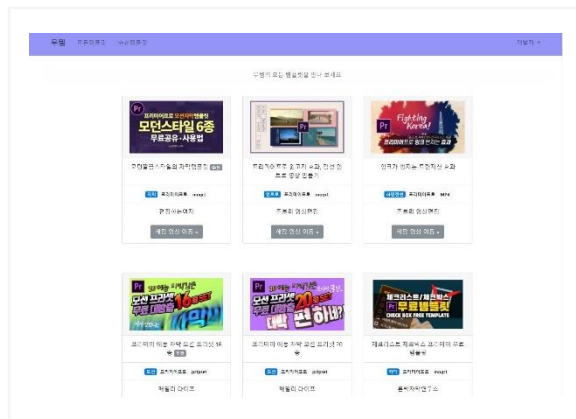
allList.jsp

```

<div class="container">
<hr>
<div class="row" align="center"
  style="word-break: break-all; word-wrap: break-word;">
  <c:forEach items=${allList} var="tVO">
    <div class="col-6 col-md-4" style="height: 460px;">
      <div class="card" style="width: 100%;">
        <div class="card-header">
          <a href="${tVO.tem_link}" target="_blank"> 
          </a>
        </div>
        <ul class="list-group list-group-flush">
          <li class="list-group-item" style="height: 70px;">${tVO.tem_name}
            <c:if test="${tVO.recommendation eq 1}">
              <span class="badge badge-secondary">추천</span>
            </c:if>
            </li>
            <li class="list-group-item"><span
              class="badge badge-primary">${tVO.tem_category}</span> <span
              class="badge badge-light">${tVO.tem_program}</span> <span
              class="badge badge-light">${tVO.tem_format}</span></li>
            <li class="list-group-item">${tVO.tem_channel}</li>
            <li class="list-group-item"><a href="${tVO.tem_link}"
              class="btn btn-secondary" role="button" target="_blank"> 해당
              영상 이동 &raquo;</a></li>
          </ul>
        </div>
      </c:forEach>
    </div>
  </div>

```

- JSTL Core 태그의 반복문을 사용하여 넘겨받은 모든 템플릿 데이터를 게시판 형식으로 출력
- 부트스트랩을 사용하여 화면 구성



```
<c:if test="${empty sessionScope.id}">
  <script type="text/javascript">
    alert("잘못된 경로입니다. 관리자모드는 로그인에 필요합니다.");
    location.href = "adminLogin"
  </script>
</c:if>
```

```
<c:forEach items="${allList}" var="tVO">
  <tr>
    <td>${tVO.tem_number}</td>
    <td>${tVO.tem_category}</td>
    <td>${tVO.tem_name}</td>
    <td>${tVO.tem_channel}</td>
    <td>${tVO.tem_format}</td>
    <td>${tVO.tem_program}</td>
    <td>${tVO.reg_date}</td>
    <td>
      <button type="button" class="btn btn-outline-dark"
        onclick="location.href='modify?tem_number=${tVO.tem_number}'">수정</button>
      <button type="button" class="btn btn-outline-dark"
        onclick="location.href='delete?tem_number=${tVO.tem_number}'">삭제</button>
    </td>
  </tr>
</c:forEach>
```

- 로그인을 하지 않고 접근하면 로그인 페이지로 이동
- 수정/삭제 버튼을 누르면 해당 템플릿의 번호를 GET 방식으로 전송하여 처리

부품 관리 목록							
관리자 권한이 필요합니다.							
번호	이름	등록일	등록자	등록처	등록일	등록자	등록처
1	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
2	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
3	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
4	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
5	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
6	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
7	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
8	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
9	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
10	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
11	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리
12	부품	2024-01-01	관리자	부품관리	2024-01-01	관리자	부품관리

#06

2019-2020

TOAST 사용자 가이드

TOAST 가이드 개선 및 멀티미디어 콘텐츠 제작

TOAST 사용자 가이드

TOAST 클라우드 서비스를 사용하시는 분들을 위한 기초 가이드 영상입니다.

개발 기간

- 2019.10.21 ~ 2020.01.02

작업 순서

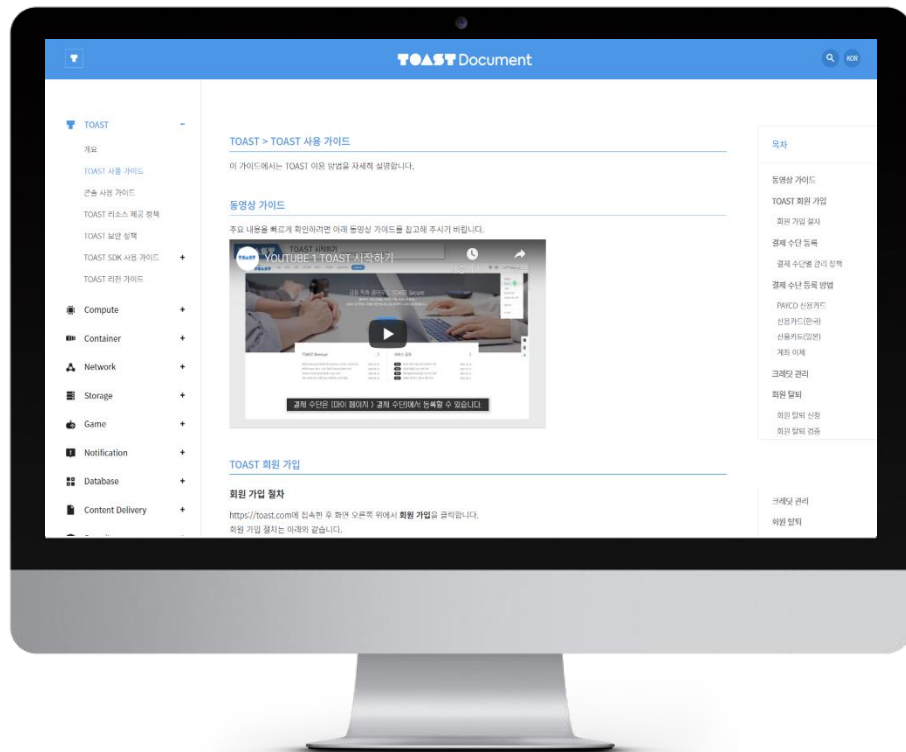
- 해당 서비스 교육
- 스크립트 및 시나리오 구성
- 영상 녹화
- 영상 편집

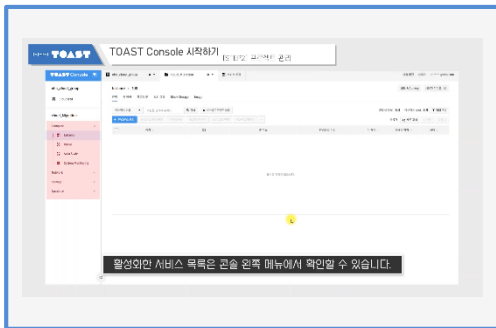
홈페이지 URL

<https://docs.toast.com/ko/TOAST/ko/user-guide/>

유튜브 URL

<https://www.youtube.com/playlist?list=PL42XJKPNDEpYOHjEXL12p9EAlt2InPlj>

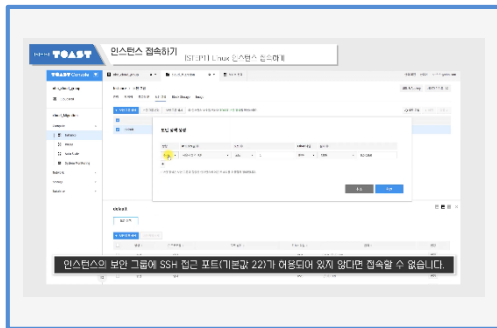




TOAST

1. TOAST 시작하기

2. TOAST Console 시작하기



Compute

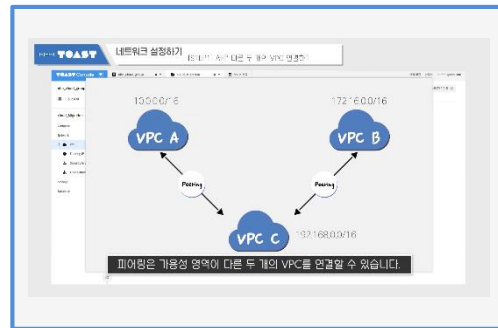
3. 클라우드 서버 구축하기

4. 사용자 OS 만들기

5. 인스턴스 부하(Load) 방지하기

6. 인스턴스 모니터링하기

7. 인스턴스 접속하기

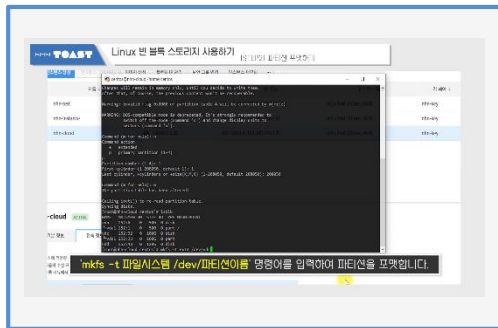


Network

8. 네트워크 구성하기

9. 네트워크 설정하기

10. 접속량 분산하기



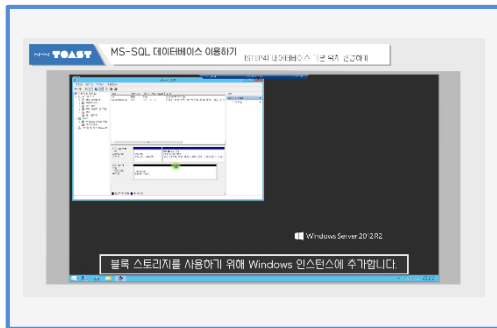
Storage

11. 가상 디스크 추가하기

12. Linux 빈 블록 스토리지 사용하기

13. Windows 빈 블록 스토리지 사용하기

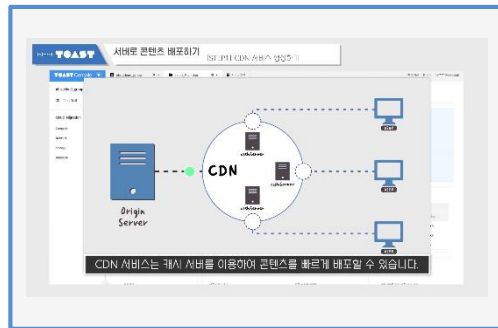
14. 스토리지 공유하기



Database

15. MYSQL 데이터베이스 이용하기

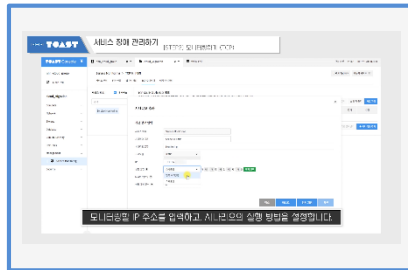
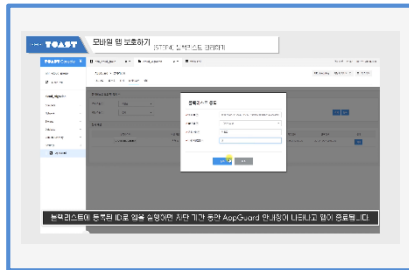
16. MS-SQL 데이터베이스 이용하기



Content Delivery

17. 서버로 콘텐츠 배포하기

18. 서비스에서 사용하는 이미지 관리하기



Security

19. 모바일 앱 보호하기

Dev Tools

20. 애플리케이션 배포하기

21. 리소스 관리하기

Management

22. 서비스 장애 관리하기

Application Service

23. API 생성 및 배포하기



2021

서희정

감사합니다.

